Trabajo Práctico: Desarrollo de Algoritmos

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	Computación
Docente:	José Oswualdo Guaman
Estudiante(s):	Jinmy Javier Vega Yaguana
Curso / Paralelo:	1^{mer} Ciclo
Fecha de entrega:	03 / 11 / 2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2.Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Estructura del trabajo (puntos a desarrollar por cada algoritmo)

EJ-003

Enunciado del algoritmo

En una empresa de transporte, diseñe un algoritmo que calcule el área de un círculo. Declare una constante simbólica PI = 3.14159 y una variable real radio. Lea el radio por teclado. Calcule el área con la expresión aritmética (PI * radio * radio) y asígnela a la variable área. Muestre por pantalla el resultado con un mensaje descriptivo. Justifique por que se usa constante simbólica y no un literal para PI. Además del cálculo principal, calcule el resto de una división adicional. Incluya un mensaje de confirmación al finalizar la operación.

Análisis del problema

Datos de entrada

Leer radio por teclado tipo real

Leer num1 , num2 por teclado tipo entero

Proceso

PI_CONST = 3.14159 Constante simbólica

Cálculo del área (PI_CONST * radio * radio) asignar en area

Calcular el resto de dividiendo y divisor (num1 % num2) asignar en resto

Datos de salida

area = el calculo del area

resto = el calculo del residuo

Diagrama de flujo

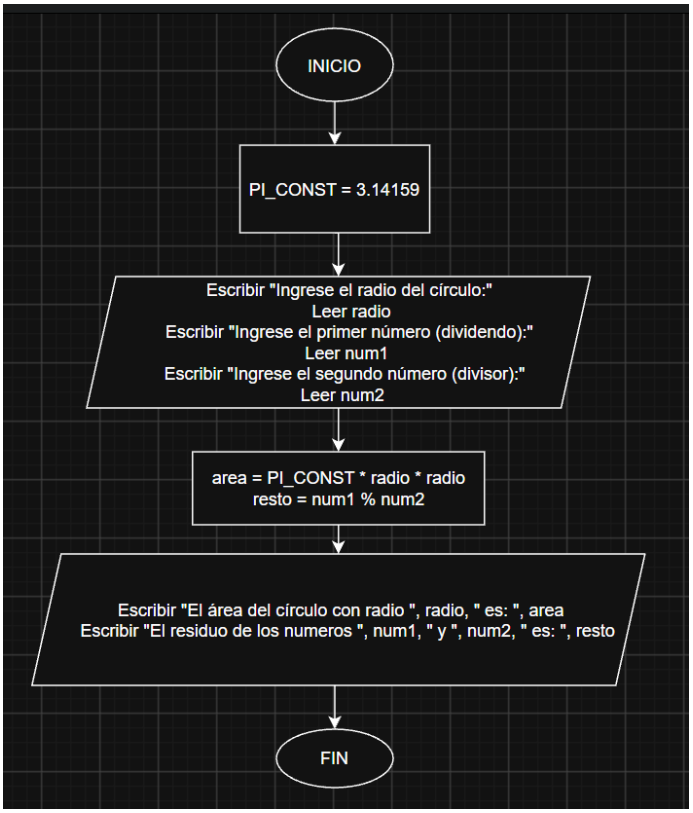


Fig. 1. Diagrama de flujo del primer ejercicio.

Pseudocódigo

```
Algoritmo seis
    // Definición de variables
    Definir radio, area Como Real
    Definir num1, num2, resto Como Entero

    // Definición de constantes
    Definir PI_CONST Como Real
    PI_CONST = 3.14159

    // Pedir por teclado al usuario
    Escribir "Ingrese el radio del círculo:"
    Leer radio
    Escribir "Ingrese el primer número (dividendo):"
    Leer num1
    Escribir "Ingrese el segundo número (divisor):"
    Leer num2

    // Cálculo del área
    area = PI_CONST * radio * radio

    // Cálculo del resto de la división
    resto = num1 % num2

    // Presentacion al usuario
    Escribir "El área del círculo con radio ", radio, " es: ", area
    Escribir "El residuo de los numeros ", num1, " y ", num2, " es: ", resto

FinAlgoritmo
```

Fig. 2. Pseudocódigo del primer ejercicio.

Prueba de escritorio

Paso	PI_CONST	radio	num1	num2	area	resto	Proceso	Pantalla
1	3.14159							
2		5					Lectura de variable radio	Ingrese el radio del círculo:
3			10				Lectura de variable num1	Ingrese el primer número (dividendo):
4				3			Lectura de variable num2	Ingrese el segundo número (divisor):
5					78.54		area = 3.14159 * 5.0 * 5.0	
6						1	resto = 10 % 3	
7							mpresión de resultados finales	El área del círculo con radio 5 es: 78.54 El residuo de los numeros 10 y 3 es: 1

Tabla 1. Primera prueba de escritorio del primer ejercicio.

Paso	PI_CONST	radio	num1	num2	area	resto	Proceso	Pantalla
1	3.14159							
2		2.5					Lectura de variable radio	Ingrese el radio del círculo:
3			17				Lectura de variable num1	Ingrese el primer número (dividendo):
4				5			Lectura de variable num2	Ingrese el segundo número (divisor):
5					19.63		area = 3.14159 * 2.5 * 2.5	
6						2	resto = 17 % 5	
7							mpresión de resultados finales	El área del círculo con radio 2.5 es: 19.63 El residuo de los numeros 17 y 5 es: 2

Tabla 2. Segunda prueba de escritorio del primer ejercicio.

Codificación en lenguaje C

seis.c > main()

```c
#include <stdio.h>

int main (){

//Definir variables
float radio , area ;
int num1 , num2 , resto ;
float PI_CONST;

//Definir constantes
PI_CONST = 3.14159;

//Pedir por teclado al usuario
printf("Ingrese el radio del circulo:\n");
scanf("%f" , &radio);

printf ("Ingrese el primer numero y segundo numero\n");
scanf("%d %d" , &num1 , &num2);

//Calculo del area
area = PI_CONST * radio * radio;

//Calculo del resto de la division
resto = num1 % num2;

//Presentacion al usuario
printf("El area del circulo con radio %.2f es %.2f \n" , radio , area);
printf("El residuo de los numeros %d y %d es %d" , num1 , num2 , resto);

return 0 ;

}
```

Fig. 3. Programación en C del primer ejercicio.

Salida del programa

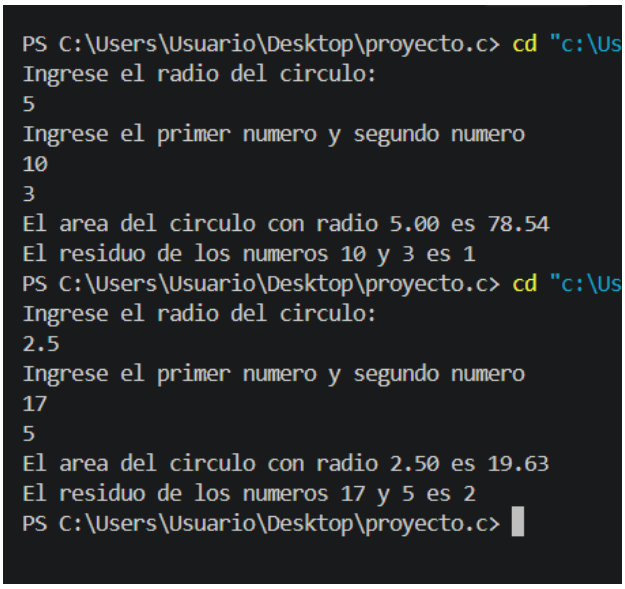


Fig. 4. Salida del programa del primer ejercicio.

Conclusiones del algoritmo

En lugar de usar π directo, definí PI_CONST para mantener el valor seguro en todo el programa y poderlo cambiar si hace falta.

En C aprendí que hay que tener cuidado al declarar variables, porque como es un lenguaje fuertemente tipado, un solo tipo de dato mal puesto no ejecuta el codigo

EJ-005

Enunciado del algoritmo

En una cooperativa, declare una variable entera a y una constante literal entera 7. Lea a por teclado. Calcule el residuo de la división (a % 7) y asigne el resultado a residuo. Evalúe la expresión lógica (residuo == 0 AND a > 0). Muestre por pantalla el mensaje 'Es múltiplo de 7' o 'No es múltiplo de 7'. Indique el tipo de dato simple de cada variable usada. Además, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecución del algoritmo.

Análisis del problema

Datos de entrada

Leer a por teclado tipo entero

Proceso

Constante literal 7 de tipo entero

Calcular el residuo de la división (a % 7) y asignar en residuo

Evaluar la expresión lógica (residuo == 0 AND a > 0) para determinar si a es múltiplo de 7 o no

Evaluar (a % 2 == 0) para determinar si a es par o impar

Datos de salida

residuo = El resultado de la división definido como entero

multiplo = Es múltiplo de 7 o No es múltiplo de 7 según la expresión lógica

par = Es par o Es impar definido como texto

Diagrama de flujo

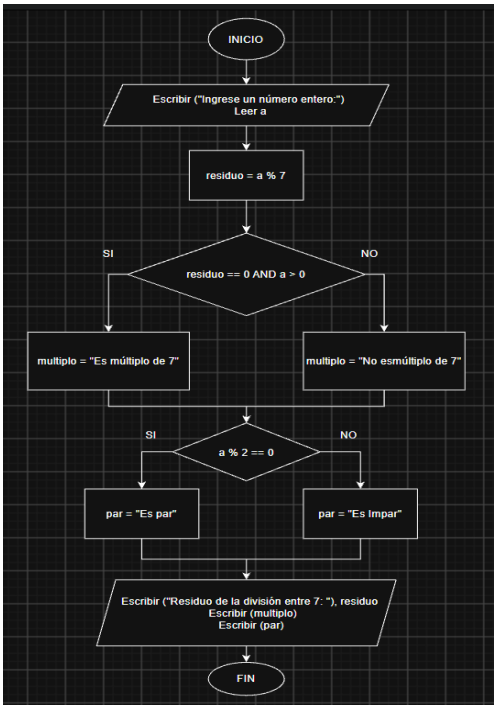


Fig. 5. Diagrama de flujo del segundo ejercicio.

Pseudocódigo

```
Algoritmo siete
    // Definición de variables
    Definir a, residuo Como Entero
    Definir multiplo , par Como Texto

    // Ingreso de varibles por teclado
    Escribir ("Ingrese un número entero:")
    Leer a

    //Calcular el residuo del valor entredo con 7
    residuo = a % 7

    //verificar si residuo es multiplo de 7
    Si (residuo == 0 Y a > 0) Entonces
        multiplo = "Es múltiplo de 7"
    Sino
        multiplo = "No es múltiplo de 7"
    FinSi

    //verificar si a es par o impar
    Si (a % 2 == 0) Entonces
        par = "Es par"
    Sino
        par = "Es impar"
    FinSi

    // Mostrar por pantalla al usuario
    Escribir ("Residuo de la división entre 7: "), residuo
    Escribir (multiplo)
    Escribir (par)

FinAlgoritmo
```

Fig. 6. Pseudocódigo del segundo ejercicio.

Prueba de escritorio

Paso	a	residuo	multiplo	par	Proceso	Pantalla
1	14				Lectura de variable a	Ingrese un número entero:
2		0			residuo = 14 % 7	
3			s múltiplo de		(0 == 0 Y 14 > 0) -> VERDADERO	
4				Es par	14 % 2 == 0 -> VERDADERO	
5					Impresión de resultados finales	Residuo de la división entre 7: 0 Es múltiplo de 7 Es par

Tabla 3. Primera prueba de escritorio del segundo ejercicio.

Paso	a	residuo	multiplo	par	Proceso	Pantalla
1	9				Lectura de variable a	Ingrese un número entero:
2		2			residuo = 9 % 7	
3			es múltiplo d		(2 == 0 Y 9 > 0) -> FALSO	
4				Es impar	9 % 2 == 0 -> FALSO	
5					Impresión de resultados finales	Residuo de la división entre 7: 2 No es múltiplo de 7 Es impar

Tabla 4. Segunda prueba de escritorio del segundo ejercicio.

Codificación en lenguaje C

SIETE.c > main()

```c
#include <stdio.h>

int main() {
    // Definición de variables
    int a, residuo;

    // Ingreso de variables por teclado
    printf("Ingrese un numero entero:\n");
    scanf("%d", &a);

    // Calcular el residuo del valor entregado con 7
    residuo = a % 7;

    // Presentación por pantalla al usuario
    printf("Residuo de la division entre 7: %d\n", residuo);

    // Verificar si es multiplo de 7
    if (residuo == 0 && a > 0) {
        printf("Es multiplo de 7\n");
    } else {
        printf("No es multiplo de 7\n");
    }

    // Verificar si a es par o impar
    if (a % 2 == 0) {
        printf("Es par\n");
    } else {
        printf("Es impar\n");
    }

    return 0;
}
```

Fig. 7. Programación en C del segundo ejercicio.

Salida del programa

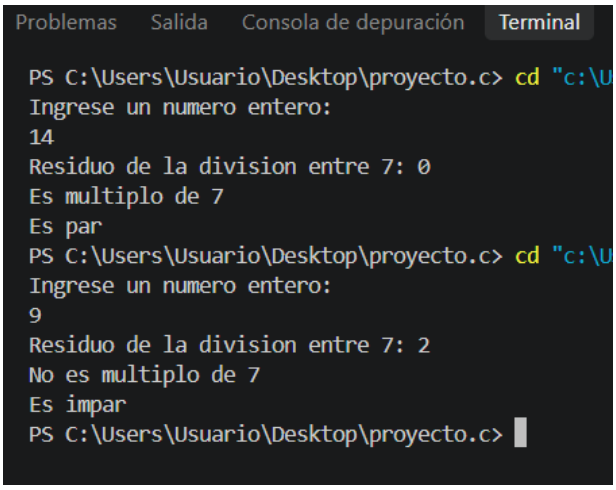


Fig. 8. Salida del programa del segundo ejercicio.

Conclusiones del algoritmo

En lugar de acumular todas las impresiones al final, preferí imprimir a medida que se realizaba cada comprobación. Esto hizo que mi código fuera mucho más ordenado y limpio.

El ejercicio me enseñó a reutilizar un mismo dato de entrada para hacer dos análisis (evaluar si es múltiplo de 7 y si es par o impar) sin necesitar algo más del usuario.

EJ-026

Enunciado del algoritmo

En un cine, construya un algoritmo que lea una medida en pulgadas (real). Use una constante simbólica PULGADA_CM = 2.54. Calcule la equivalencia en centímetros con la expresión aritmética (pulgadas * PULGADA_CM) y asígnela a cm. Evalúe la expresión relacional (cm > 0). Muestre por pantalla 'X pulgadas equivalen a Y cm'. Además del cálculo principal, calcule el producto del resultado por PI. Agregue un mensaje indicando el tipo de operación realizada.

Análisis del problema

Datos de entrada

Leer pulgadas por teclado tipo real

Proceso

PULGADA_CM = 2.54 Constante simbólica de tipo real

PI = 3.14159 Constante simbólica de tipo real

Calcular la conversión a centímetros (pulgadas * PULGADA_CM) y asignar en cm

Evaluar la expresión relacional (cm > 0) para verificar que sea mayor que 0

Calcular el producto del resultado por PI (cm * PI) y asignar en producto_pi

Datos de salida

Texto presentado lo convertido ("X pulgadas equivalen a Y cm")

cm = El resultado de la conversión a centímetros definido como real

producto_pi = El resultado de multiplicar los centímetros por PI definido como real

Diagrama de flujo

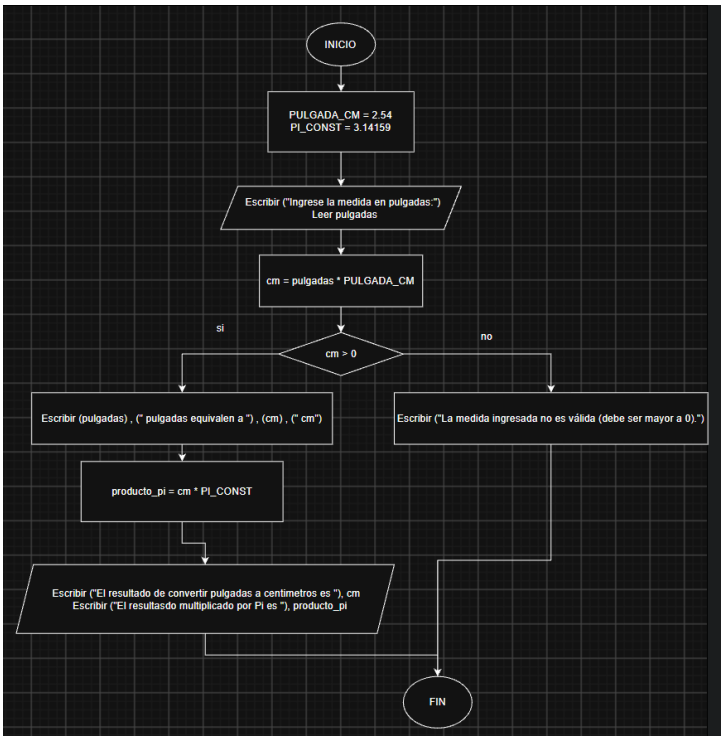


Fig. 9. Diagrama de flujo del tercer ejercicio.

Pseudocódigo

```
Algoritmo ocho

    // Definición de variables
    Definir pulgadas, cm, producto_pi Como Real
    Definir PULGADA_CM, PI_CONST Como Real

    // Definición de constantes
    PULGADA_CM = 2.54
    PI_CONST = 3.14159

    // PEDIR DATOS AL USUARIO
    Escribir ("Ingrese la medida en pulgadas:")
    Leer pulgadas

    // Calcular equivalencia en centímetros
    cm = pulgadas * PULGADA_CM

    // Evaluar la expresión
    Si (cm > 0) Entonces
        Escribir (pulgadas) , (" pulgadas equivalen a ") , (cm) , (" cm")

        // Calcular el producto por PI solo si es válido
        producto_pi = cm * PI_CONST

        // Presentar al usuario
        Escribir ("El resultado de convertir pulgadas a centimetros es "), cm
        Escribir ("El resultado multiplicado por Pi es "), producto_pi
    Sino
        Escribir ("La medida ingresada no es válida (debe ser mayor a 0).")
    FinSi

FinAlgoritmo
```

Fig. 10. Pseudocódigo del tercer ejercicio.

Prueba de escritorio

Paso	PULGADA_CM	PI_CONST	pulgadas	cm	producto_pi	Proceso	Pantalla
1	2.54	3.14159					
2			10			Lectura de variable pulgadas	Ingrese la medida en pulgadas:
3				25.4		cm = 10.0 * 2.54	
4					79.8	VERDADERO -> producto_pi = 25.4 *	
5						Impresión de resultados finales	10 pulgadas equivalen a 25.4 cm El resultado de convertir pulgadas a centimetros es 25.4 El resultado multiplicado por Pi es 79.8

Tabla 5. Primera prueba de escritorio del tercer ejercicio.

Paso	PULGADA_CM	PI_CONST	pulgadas	cm	producto_pi	Proceso	Pantalla
1	2.54	3.14159					
2			-5			Lectura de variable pulgadas	Ingrese la medida en pulgadas:
3				-12.7		cm = -5.0 * 2.54	
4						cm > 0 -> FALSO	La medida ingresada no es válida (debe ser mayor a 0).

Tabla 6. Segunda prueba de escritorio del tercer ejercicio.

Codificación en lenguaje C

OCHOOO.C > main()

```c
#include <stdio.h>

int main() {
    // Definición de variables
    float pulgadas, cm, producto_pi;
    float PULGADA_CM, PI_CONST;

    // Definición de constantes
    PULGADA_CM = 2.54;
    PI_CONST = 3.14159;

    // PEDIR DATOS AL USUARIO
    printf("Ingrese la medida en pulgadas:\n");
    scanf("%f", &pulgadas);

    // Calcular equivalencia en centímetros
    cm = pulgadas * PULGADA_CM;

    // Evaluar la expresión
    if (cm > 0) {
        printf("%.2f pulgadas equivalen a %.2f cm\n", pulgadas, cm);

        // Calcular el producto por PI solo si es válido
        producto_pi = cm * PI_CONST;

        // Presentar al usuario
        printf("El resultado de convertir pulgadas a centimetros es %.2f\n", cm);
        printf("El resultado multiplicado por Pi es %.2f\n", producto_pi);
    } else {
        printf("La medida ingresada no es valida (debe ser mayor a 0).\n");
    }

    return 0;
}
```

Fig. 11. Programación en C del tercer ejercicio.

Salida del programa

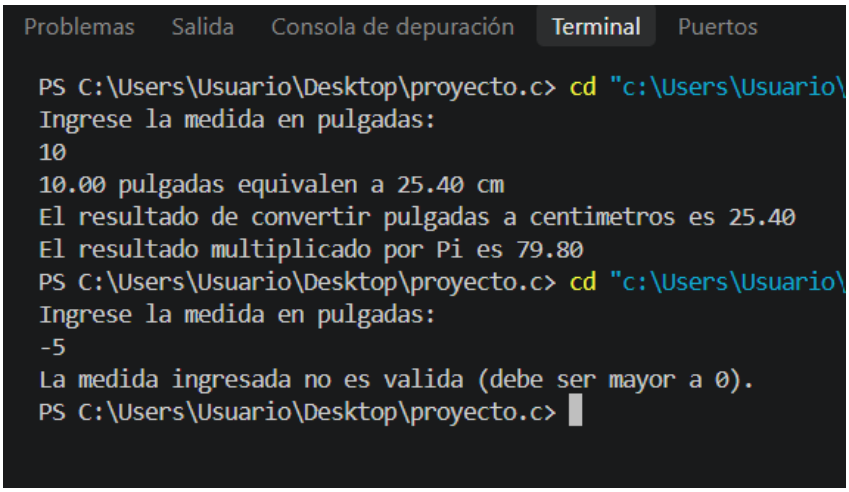


Fig. 12. Salida del programa del tercer ejercicio.

Conclusiones del algoritmo

Usar la condición (cm > 0) me ayudó a controlar que el programa solo muestre resultados si cm era mayor a 0 y evitando trabajar con valores incorrectos o negativos.

Este ejercicio me permitió comprender como se transforman cm a pulgadas con una simple constante y una multiplicación logrando que mi algoritmo se calcule bien.

EJ-047

Enunciado del algoritmo

En una librería, construya un algoritmo que lea una distancia en millas (real). Use una constante simbolica MILLA_KM = 1.60934. Calcule la equivalencia en kilómetros con la expresión aritmética (millas * MILLA_KM) y asígnela a km. Evalúe la expresión relacional (km > 0). Muestre por pantalla 'X millas equivalen a Y km'. Además del cálculo principal, calcule el logaritmo en base 10 del resultado. Muestre al final los valores de todas las variables auxiliares.

Análisis del problema

Datos de entrada

Leer millas por teclado de tipo real.

Proceso

Constante MILLA_KM = 1.60934 tipo real

Calcular la conversión a kilómetros (millas * MILLA_KM) y asignar a km.

Evaluar la condición (km > 0) para ver si la distancia es mayor a 0 y seguir con los demás procesos.

Calcular el logaritmo en base 10 log10 = lnx/ln10 y asignar en logaritmo_km.

Datos de salida

Texto = "X millas equivalen a Y km"

km: El resultado de la conversión a kilómetros definido como real.

logaritmo_km: El resultado de calcular el logaritmo en base 10 de los kilómetros definido como real.

Diagrama de flujo

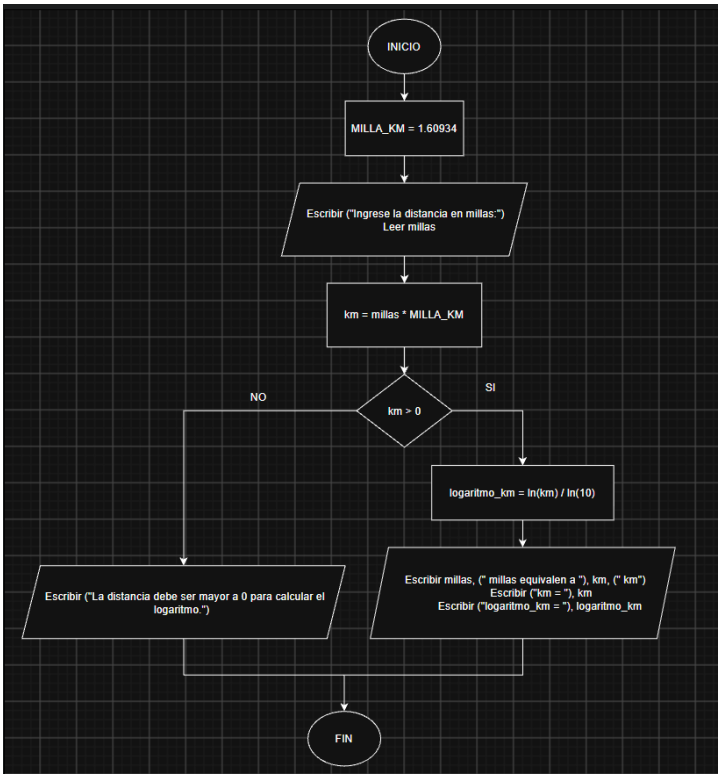


Fig. 13. Diagrama de flujo del cuarto ejercicio.

Pseudocódigo

```
Algoritmo nueve
    // Definicion de variables
    Definir millas, km, logaritmo_km Como Real
    Definir MILLA_KM Como Real

    // Definicion de constantes
    MILLA_KM = 1.60934

    //Lectura por teclado
    Escribir ("Ingrese la distancia en millas:")
    Leer millas

    //Proceso
    km = millas * MILLA_KM

    // Evaluar si la distancia es mayor a 0 para seguir con los demas procesos
    Si (km > 0) Entonces
        logaritmo_km = ln(km) / ln(10)

        // Datos de salida
        Escribir millas, (" millas equivalen a "), km, (" km")
        Escribir ("km = "), km
        Escribir ("logaritmo_km = "), logaritmo_km
    Sino
        Escribir ("La distancia debe ser mayor a 0 para calcular el logaritmo.")
    FinSi
FinAlgoritmo
```

Fig. 14. Pseudocódigo del cuarto ejercicio.

Prueba de escritorio

Paso	MILLA_KM	millas	km	logaritmo_km	Proceso	Pantalla
1	1.60934					
2		10			Lectura de variable millas	Ingrese la distancia en millas:
3			16.09		km = 10.0 * 1.60934	
4				1.21	km > 0 -> VERDADERO -> logaritmo_km = ln(16.09) / ln(10)	
5					Impresión de resultados finales	10 millas equivalen a 16.09 km km = 16.09 logaritmo_km = 1.21

Tabla 7. Primera prueba de escritorio del cuarto ejercicio.

Paso	MILLA_KM	millas	km	logaritmo_km	Proceso	Pantalla
1	1.60934					
2		-5			Lectura de variable millas	Ingrese la distancia en millas:
3			-8.05		km = -5.0 * 1.60934	
4					km > 0 -> FALSO	La distancia debe ser mayor a 0 para calcular el logaritmo.

Tabla 8. Segunda prueba de escritorio del cuarto ejercicio.

Codificación en lenguaje C

C NUEVE.C > main()

```c
#include <stdio.h>
#include <math.h>

int main() {
    // Definicion de variables
    float millas, km, logaritmo_km;

    // Definicion de constante
    const float MILLA_KM = 1.60934;

    // Lectura por teclado
    printf("Ingrese la distancia en millas: ");
    scanf("%f", &millas);

    // Proceso
    km = millas * MILLA_KM;

    // Evaluar si la distancia es mayor a 0 para seguir con los demas procesos
    if (km > 0) {
        logaritmo_km = log10(km);

        // Datos de salida
        printf("%.2f millas equivalen a %.2f km\n", millas, km);
        printf("km = %.4f\n", km);
        printf("logaritmo_km = %.4f\n", logaritmo_km);
    } else {
        printf("La distancia debe ser mayor a 0 para calcular el logaritmo.\n");
    }

    return 0;
}
```

Fig. 15. Programación en C del cuarto ejercicio.

Salida del programa

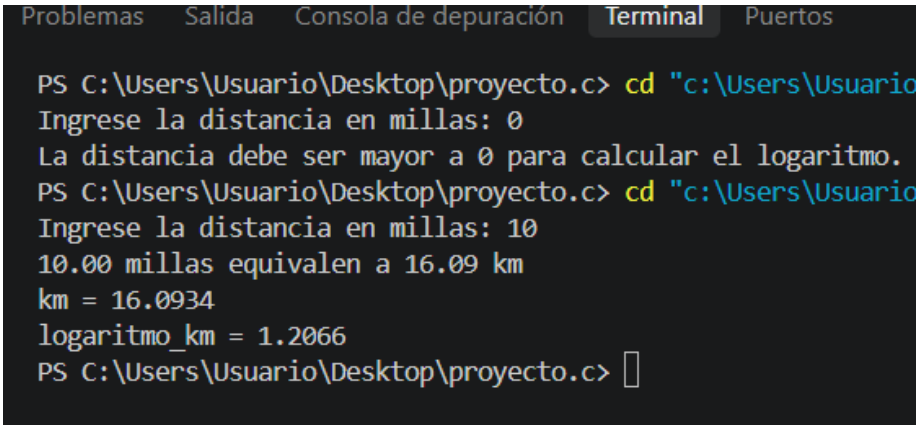


Fig. 16. Salida del programa del cuarto ejercicio.

Conclusiones del algoritmo

Aprendí a utilizar la fórmula para calcular el logaritmo en base 10 dentro de un algoritmo tanto en legajé C y en un seudocódigo

Concluí que para calcular un logaritmo en base 10 la cantidad debe ser mayor a cero, gracias a esto mi código se ejecutó de una buena manera

EJ-050

Enunciado del algoritmo

En una cooperativa, construya un algoritmo que lea una altura en pies (real). Use una constante simbólica PIE_METRO = 0.3048. Calcule la equivalencia en metros con la expresión aritmética (pies * PIE_METRO) y asígnela a metros. Evalúe la expresión relacional (metros > 0). Muestre por pantalla 'X pies equivalen a Y metros'. Además, determine si el resultado es par o impar (cuando aplique). Incluya mensajes informativos durante la ejecución del algoritmo.

Análisis del problema

Datos de entrada

Leer pies por teclado tipo real

Proceso

PIE_METRO = 0.3048 Constante simbólica de tipo real

Calcular la conversión a metros (pies * PIE_METRO) y asignar en metros

Evaluar la condición (metros > 0) para verificar que sea mayor que 0

Verificar si el número es par o impar con la condición (metros % 2 == 0)

Datos de salida

Texto presentado lo convertido ("X pies equivalen a Y metros")

metros = El resultado de la conversión a metros definido como real

La verificación si el numero ingresado es par o impar

Diagrama de flujo

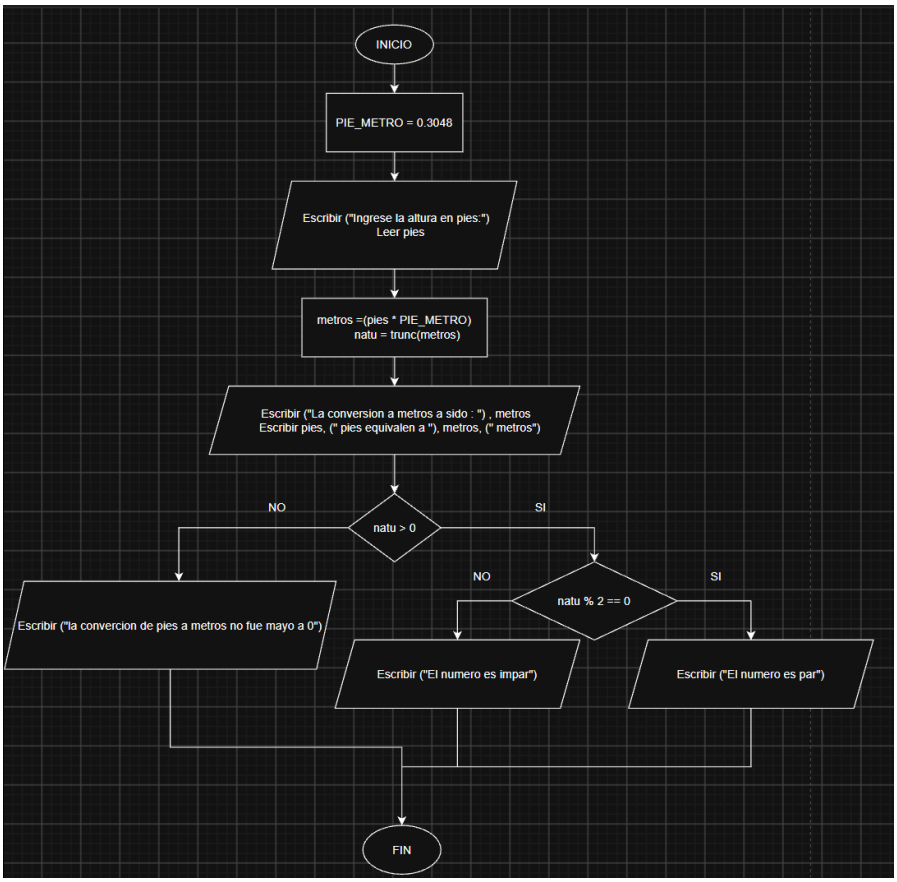


Fig. 17. Diagrama de flujo del quinto ejercicio.

Pseudocódigo

```
Algoritmo dies
    // DEFINICION DE VARIABLES
    Definir PIE_METRO Como Real
    Definir pies Como Real
    Definir metros Como real
    Definir natu Como Entero
    //CONSTANTE
    PIE_METRO = 0.3048

    //LECTURA POR TECLADO
    Escribir ("Ingrese la altura en pies:")
    Leer pies

    // PROCESO
    metros =(pies * PIE_METRO)
    natu = trunc(metros)
    //PRESENTACION
    Escribir ("La conversion a metros a sido : ") , metros
    Escribir pies, (" pies equivalen a "), metros, (" metros")


    Si (natu > 0) Entonces
        Si (natu % 2 == 0) Entonces
            Escribir ("El numero es par")
        Sino
            Escribir ("El numero es impar")
        FinSi
    Sino
        Escribir ("la convercion de pies a metros no fue mayo a 0")
    FinSi
FinAlgoritmo
```

Fig. 18. Pseudocódigo del quinto ejercicio.

Prueba de escritorio

Paso	PIE_METRO	pies	metros	natu	Proceso	Pantalla
1	0.3048					
2		20			Lectura de variable pies	Ingrese la altura en pies:
3			6.1		metros = 20.0 * 0.3048	
4				6	natu = trunc(6.10)	
5					6 > 0 Y 6 % 2 == 0 -> VERDADERO	La conversion a metros a sido : 6.10 20 pies equivalen a 6.10 metros El numero es par

Tabla 9. Primera prueba de escritorio del quinto ejercicio.

Paso	PIE_METRO	pies	metros	natu	Proceso	Pantalla
1	0.3048					
2		10			Lectura de variable pies	Ingrese la altura en pies:
3			3.05		metros = 10.0 * 0.3048	
4				3	natu = trunc(3.05)	
5					3 > 0 -> VERDADERO, 3 % 2 == 0 -> FALSO	La conversion a metros a sido : 3.05 10 pies equivalen a 3.05 metros El numero es impar

Tabla 10. Segunda prueba de escritorio del quinto ejercicio.

Codificación en lenguaje C

DIES.C > main()

```c
#include <stdio.h>

int main() {
    // DEFINICION VARIABLES
    float pies , metros;
    //CONSTANTE
    float PIE_METRO = 0.3048;

    // LECTURA POR TECLADO
    printf("Ingrese la altura en pies: ");
    scanf("%f", &pies);

    // PROCESO
    metros = pies * PIE_METRO;

    // PRESENTACION
    printf("La conversion a metros ha sido: %.2f\n", metros);
    printf("%.2f pies equivalen a %.2f metros\n", pies, metros);

    // CONDICIONES PARA PRESENTAR AL USUARIO
    if (metros > 0) {
        //APLICACION DE UN INT PARA SOLO TOMAR LA PARTE ENTERA
        if ((int)metros % 2 == 0) {
            printf("El numero es par\n");
        } else {
            printf("El numero es impar\n");
        }
    } else {
        printf("La conversion de pies a metros no fue mayor a 0\n");
    }

    return 0;
}
```

Fig. 19. Programación en C del quinto ejercicio.

Salida del programa

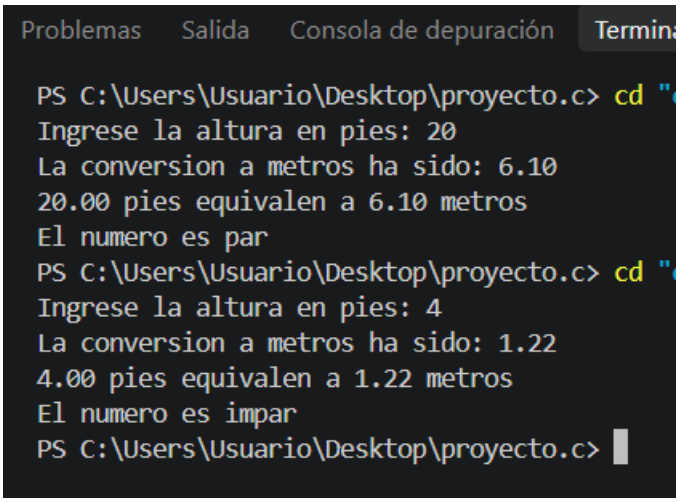


Fig. 20. Salida del programa del quinto ejercicio.

Conclusiones del algoritmo

Aprendí que, tanto en PSeInt como en C, la operación de residuo % no funciona con números reales. Para calcular si un número es par o impar, las dos variables deben de ser tipo entero; si no el sistema da error porque no existe igualdad en las variables.

En PSeInt, como mi variable metros tenía decimales, creé una variable llamada natu definida como Entero y le asigné Trunc(metros) para solo tener la parte entera y aplicar el módulo, en lenguaje C no necesité declarar otra variable lo hice directo usando (int)metros, eso convierte el valor a entero para poder calcular el módulo.

EJ-118

Enunciado del algoritmo

En un gimnasio, construya un algoritmo que lea un volumen en litros (real). Use la constante literal 1000. Calcule la equivalencia en mililitros con la expresión aritmética (litros * 1000) y asígnela a ml. Evalúe la expresión relacional (ml >= 0). Muestre por pantalla 'X litros equivalen a Y ml'. Adicionalmente, calcule el porcentaje del resultado respecto a un valor de referencia. Justifique el tipo de dato simple elegido para cada variable.

Análisis del problema

Datos de entrada

Leer litros por teclado tipo real

Proceso

MIL =1000 constante literal de tipo entero

REFERENCIA = 2000.0 Constante simbólica de tipo real (valor de referencia que pide el problema)

Calcular la conversión a mililitros (litros * 1000) y asignar en ml

Evaluar la condición (ml >= 0) para verificar que sea mayor o igual a 0

Calcular el porcentaje utilizando la referencia ((ml / REFERENCIA) * 100) y asignar en porcentaje

Datos de salida

Texto presentado lo convertido ("X litros equivalen a Y ml")

ml = El resultado de la conversión a mililitros definido como real

porcentaje = El porcentaje calculado respecto a la referencia definido como real

Diagrama de flujo

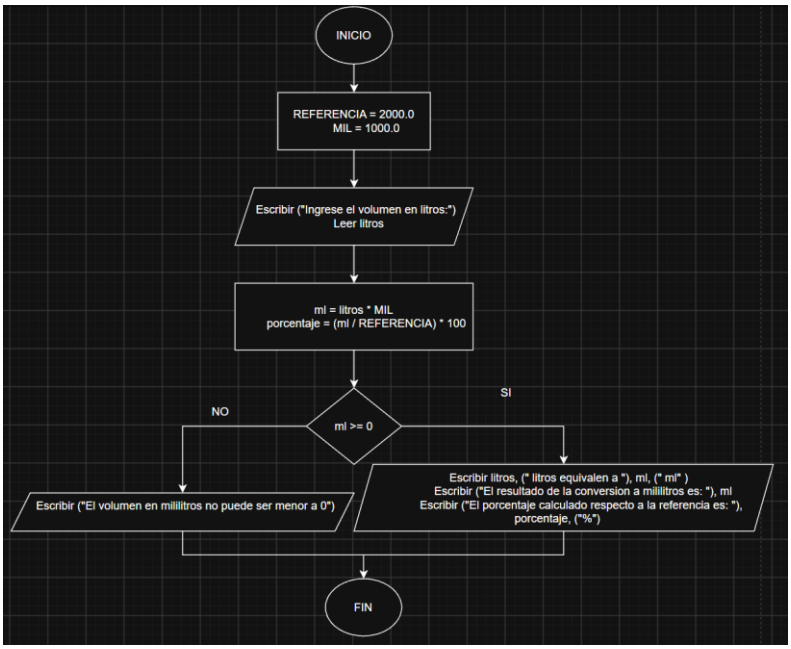


Fig. 21. Diagrama de flujo del sexto ejercicio.

Pseudocódigo

```
Algoritmo once
    // DEFINICION DE VARIABLES
    Definir litros , MIL Como Real
    Definir ml Como Real
    Definir porcentaje Como Real
    Definir REFERENCIA Como Real

    // CONSTANTE
    REFERENCIA = 2000.0
    MIL = 1000.0

    // LECTURA POR TECLADO
    Escribir ("Ingrese el volumen en litros:")
    Leer litros

    // PROCESO
    ml = litros * MIL
    porcentaje = (ml / REFERENCIA) * 100

    //PRESENTACION AL USUARIO
    Si (ml ≥ 0) Entonces
        Escribir litros, (" litros equivalen a "), ml, (" ml" )
        Escribir ("El resultado de la conversion a mililitros es: "), ml
        Escribir ("El porcentaje calculado respecto a la referencia es: "), porcentaje, ("%")
    Sino
        Escribir ("El volumen en mililitros no puede ser menor a 0")
    FinSi
FinAlgoritmo
```

Fig. 22. Pseudocódigo del sexto ejercicio.

Prueba de escritorio

Paso	REFERENCIA	MIL	litros	ml	porcentaje	Proceso	Pantalla
1	2000	1000					
2			2.5			Lectura de variable litros	Ingrese el volumen en litros:
3				2500		ml = 2.5 * 1000.0	
4					125	porcentaje = (2500.0 / 2000.0) * 100	
5						ml >= 0 -> VERDADERO	2.5 litros equivalen a 2500 ml El resultado de la conversion a mililitros es: 2500 El porcentaje calculado respecto a la referencia es: 125%

Tabla 11. Primera prueba de escritorio del sexto ejercicio.

Paso	REFERENCIA	MIL	litros	ml	porcentaje	Proceso	Pantalla
1	2000	1000					
2			-1			Lectura de variable litros	Ingrese el volumen en litros:
3				-1000		ml = -1.0 * 1000.0	
4					-50	porcentaje = (-1000.0 / 2000.0) * 100	
5						ml >= 0 -> FALSO	El volumen en mililitros no puede ser menor a 0

Tabla 12. Segunda prueba de escritorio del sexto ejercicio.

Codificación en lenguaje C

C ONCE.C > main()

```c
#include <stdio.h>

int main() {
    // DEFINICION DE VARIABLES
    float litros, ml, porcentaje;
    const float MIL = 1000.0;
    const float REFERENCIA = 2000.0;

    // LECTURA POR TECLADO
    printf("Ingrese el volumen en litros: ");
    scanf("%f", &litros);

    // PROCESO
    ml = litros * MIL;
    porcentaje = (ml / REFERENCIA) * 100;

    // PRESENTACION AL USUARIO
    if (ml >= 0) {
        printf("%.2f litros equivalen a %.2f ml\n", litros, ml);
        printf("El resultado de la conversion a mililitros es: %.2f\n", ml);
        printf("El porcentaje calculado respecto a la referencia es: %.2f%%\n", porcentaje);
    } else {
        printf("El volumen en mililitros no puede ser menor a 0\n");
    }

    return 0;
}
```

Fig. 23. Programación en C del sexto ejercicio.

Salida del programa

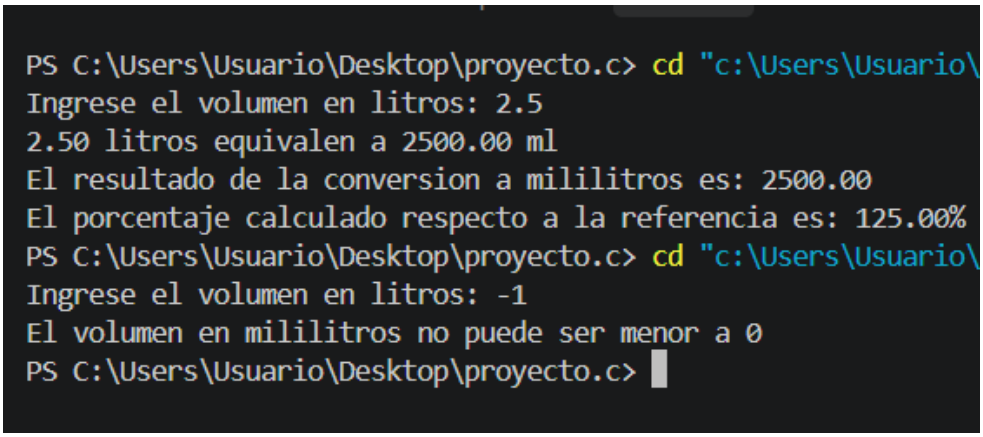


Fig. 24. Salida del programa del sexto ejercicio.

Conclusiones del algoritmo

Aprendí a multiplicar los litros por la constante 1000 esto permite transformar a mililitros, y al definirla como real permite que sea más precisa.

Aprendí a usar la constante 2000.0 para calcular mediante una división qué porcentaje total representa el volumen en mililitros.

EJ-128

Enunciado del algoritmo

En una gasolinera, diseñe un algoritmo que lea una velocidad en mph (real). Use una constante simbólica FACTOR = 1.60934. Calcule la equivalencia en km/h con la expresion aritmética (mph * FACTOR) y asígnela a kmh. Evalúe la expresion relacional (kmh > 0). Muestre por pantalla 'X mph equivalen a Y km/h'. Ademas del cálculo principal, determine el valor máximo de las variables utilizadas. Justifique el tipo de dato simple elegido para cada variable.

Análisis del problema

Datos de entrada

Leer mph por teclado tipo real

Proceso

FACTOR = 1.60934 Constante simbólica de tipo real

Calcular la conversión a km/h (mph * FACTOR) y asignar en kmh

Evaluar la condición (kmh > 0) para verificar si la velocidad es mayor que 0

Determinar el valor máximo entre las variables mph y kmh utilizando mph > kmh asignar maximo = mph y si es falso maximo = kmh)

Datos de salida

Texto presentado lo convertido ("X mph equivalen a Y km/h")

kmh = El resultado de la conversión a km/h definido como real

maximo = El valor mayor entre las variables de velocidad definido como real

Diagrama de flujo



Fig. 25. Diagrama de flujo del séptimo ejercicio.

Pseudocódigo

```
Algoritmo doce
    // DEFINICION DE VARIABLES
    Definir mph Como Real
    Definir kmh Como Real
    Definir maximo Como Real
    Definir FACTOR Como Real

    // CONSTANTE
    FACTOR = 1.60934

    // LECTURA POR TECLADO
    Escribir ("Ingrese la velocidad en mph:")
    Leer mph

    // PROCESO
    kmh = mph * FACTOR

    // DETERMINAR VALOR MAXIMO
    Si (mph > kmh) Entonces
        maximo = mph
    Sino
        maximo = kmh
    FinSi

    // PRESENTACION AL USUARIO
    Si (kmh > 0) Entonces
        Escribir mph, (" mph equivalen a "), kmh, (" km/h")
        Escribir ("El resultado de la conversion a km/h es: "), kmh
        Escribir ("El valor maximo registrado es: "), maximo
    Sino
        Escribir ("La velocidad en km/h no es mayor a 0")
    FinSi
FinAlgoritmo
```

Fig. 26. Pseudocódigo del séptimo ejercicio.

Prueba de escritorio

Paso	FACTOR	mph	kmh	maximo	Proceso	Pantalla
1	1.60934					
2		60			Lectura de variable mph	Ingrese la velocidad en mph:
3			96.56		kmh = 60.0 * 1.60934	
4				96.56	60.0 > 96.56 -> FALSO -> maximo = 96.56	
5					kmh > 0 -> VERDADERO	60 mph equivalen a 96.56 km/h El resultado de la conversion a km/h es: 96.56 El valor maximo registrado es: 96.56

Tabla 13. Primera prueba de escritorio del séptimo ejercicio.

Paso	FACTOR	mph	kmh	maximo	Proceso	Pantalla
1	1.60934					
2		-10			Lectura de variable mph	Ingrese la velocidad en mph:
3			-16.09		kmh = -10.0 * 1.60934	
4				-10	-10.0 > -16.09 -> VERDADERO -> maximo = -10.0	
5					kmh > 0 -> FALSO	La velocidad en km/h no es mayor a 0

Tabla 14. Segunda prueba de escritorio del séptimo ejercicio.

Codificación en lenguaje C

C DOCE.C > main()

```c
#include <stdio.h>

int main() {
    // DEFINICION DE VARIABLES
    float mph, kmh, maximo;
    const float FACTOR = 1.60934;

    // LECTURA POR TECLADO
    printf("Ingrese la velocidad en mph: ");
    scanf("%f", &mph);

    // PROCESO
    kmh = mph * FACTOR;

    // DETERMINAR VALOR MAXIMO
    if (mph > kmh) {
        maximo = mph;
    } else {
        maximo = kmh;
    }

    // PRESENTACION AL USUARIO
    if (kmh > 0) {
        printf("%.2f mph equivalen a %.2f km/h\n", mph, kmh);
        printf("El resultado de la conversion a km/h es: %.2f\n", kmh);
        printf("El valor maximo registrado es: %.2f\n", maximo);
    } else {
        printf("La velocidad en km/h no es mayor a 0\n");
    }

    return 0;
}
```

Fig. 27. Programación en C del séptimo ejercicio.

Salida del programa

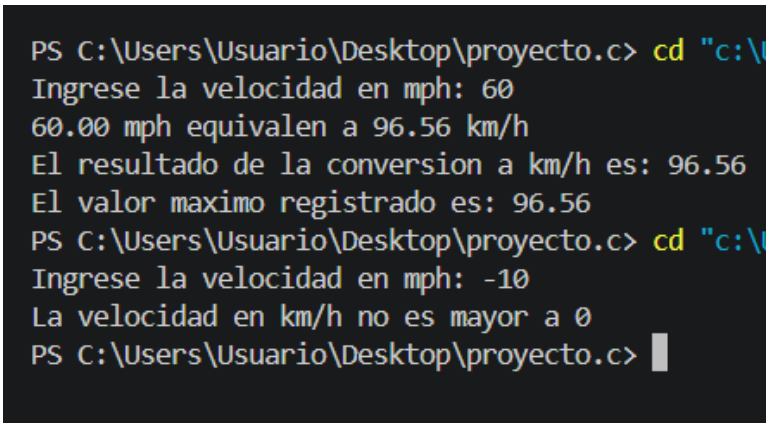


Fig. 28. Salida del programa del séptimo ejercicio.

Conclusiones del algoritmo

Aprendí que multiplicando la velocidad en mph por la constante 1.60934, obtengo el valor aproximado en km/h conservando algunos decimales para mejorar su precisión.

Comprendí cómo usar una estructura condicional Si...Sino o if-else en C para comparar mph con kmh y guardar el valor más alto dentro de la variable máximo.

EJ-134

Enunciado del algoritmo

En una florería, diseñe un algoritmo que lea un tamaño en bytes (entero). Use la constante literal 1024. Calcule la equivalencia en KB con la expresion aritmética (bytes / 1024) y asígnela a kb. Evalúe la expresion lógica (bytes >= 0). Muestre por pantalla 'X bytes equivalen a Y KB'. Justifique el tipo de dato de cada variable. Adicionalmente, redondee el resultado a 2 decimales. Agregue una validación para asegurar que los datos sean positivos.

Análisis del problema

Datos de entrada

Leer bytes por teclado tipo entero

Proceso

DIVI = 1024 Constante literal de tipo real

Evaluar la condición (bytes >= 0) para validar que la cantidad ingresada sea positiva o cero

Calcular la equivalencia en KB (bytes / DIVI) y asignar en kb como valor real

Aplicar redondeo a 2 decimales para la presentación en pantalla usando Redon(kb * 100) / 100 asignar en redondeo

Datos de salida

Texto presentado lo convertido ("X bytes equivalen a Y KB")

kb = El resultado de la conversión a kilobytes definido como real

redondeo = el resultado del redondeo de kb

Diagrama de flujo

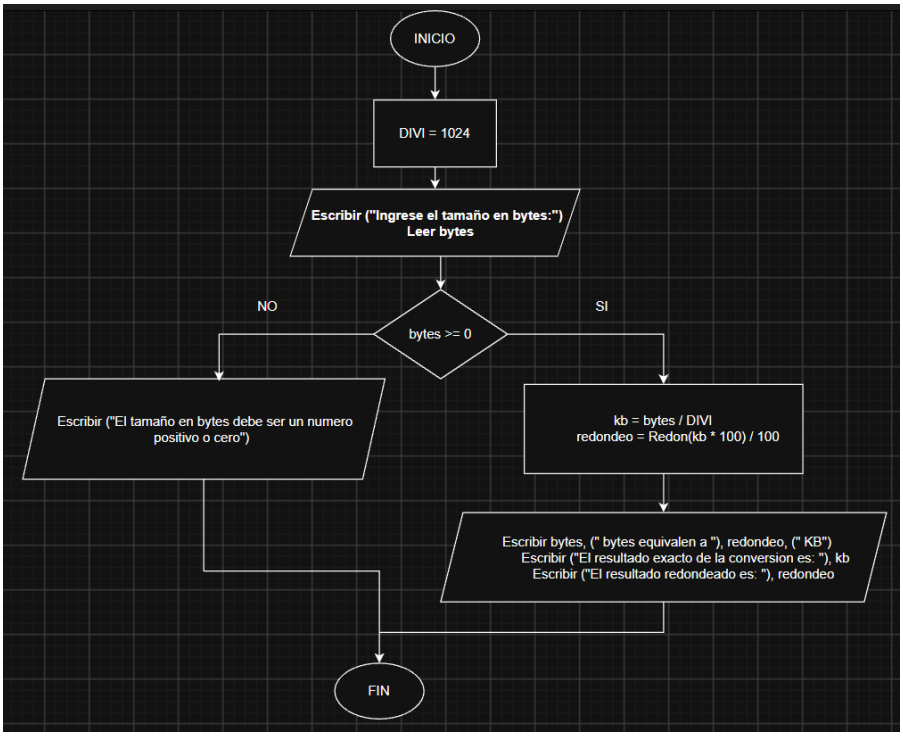


Fig. 29. Diagrama de flujo del octavo ejercicio.

Pseudocódigo

```
Algoritmo trece
    // DEFINICION DE VARIABLES
    Definir bytes Como Entero
    Definir kb Como Real
    Definir redondeo Como Real
    Definir DIVI Como real

    // CONSTANTE
    DIVI = 1024

    // LECTURA POR TECLADO
    Escribir ("Ingrese el tamaño en bytes:")
    Leer bytes

    // PROCESO Y VALIDACION
    Si (bytes ≥ 0) Entonces
        kb = bytes / DIVI
        redondeo = Redon(kb * 100) / 100

        // PRESENTACION AL USUARIO
        Escribir bytes, (" bytes equivalen a "), redondeo, (" KB")
        Escribir ("El resultado exacto de la conversion es: "), kb
        Escribir ("El resultado redondeado es: "), redondeo
    Sino
        Escribir ("El tamaño en bytes debe ser un numero positivo o cero")
    FinSi
FinAlgoritmo
```

Fig. 30. Pseudocódigo del octavo ejercicio.

Prueba de escritorio

Paso	DIVI	bytes	kb	redondeo	Proceso	Pantalla
1	1024					
2		2050			Lectura de variable bytes	Ingrese el tamaño en bytes:
3			2.002		kb = 2050 / 1024	
4				2	redondeo = Redon(2.0020 * 100) / 100	
5					bytes >= 0 -> VERDADERO	2050 bytes equivalen a 2 KB El resultado exacto de la conversion es: 2.0020 El resultado redondeado es: 2

Tabla 15. Primera prueba de escritorio del octavo ejercicio.

Paso	DIVI	bytes	kb	redondeo	Proceso	Pantalla
1	1024					
2		-500			Lectura de variable bytes	Ingrese el tamaño en bytes:
3					bytes >= 0 -> FALSO	El tamaño en bytes debe ser un numero positivo o cero

Tabla 16. Segunda prueba de escritorio del octavo ejercicio.

Codificación en lenguaje C

TRECE.C > main()

```c
#include <stdio.h>
//LIBRERIA PARA PODER REDONDEAR EL RESULTADO
#include <math.h>

int main() {
    // DEFINICION DE VARIABLES
    int bytes;
    float kb;
    float redondeo;
    const float DIVI = 1024.0;

    // LECTURA POR TECLADO
    printf("Ingrese el size en bytes: ");
    scanf("%d", &bytes);

    // PROCESO Y VALIDACION
    if (bytes >= 0) {

        kb = bytes / DIVI;
        redondeo = round(kb * 100.0) / 100.0;

        // PRESENTACION AL USUARIO
        printf("%d bytes equivalen a %.2f KB\n", bytes, redondeo);
        printf("El resultado exacto de la conversion es: %f\n", kb);
        printf("El resultado redondeado es: %.2f\n", redondeo);
    } else {
        printf("El size en bytes debe ser un numero positivo o cero\n");
    }

    return 0;
}
```

Fig. 31. Programación en C del octavo ejercicio.

Salida del programa

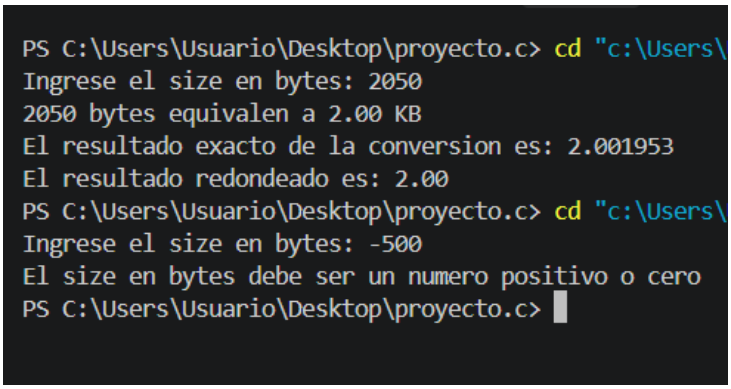


Fig. 32. Salida del programa del octavo ejercicio.

Conclusiones del algoritmo

Aprendí que para aplicar funciones matemáticas de redondeo como round () en lenguaje C, es importante incluir la librería <math.h>, la cual permite hacer estas operaciones.

Gracias a la condición que me daba el problema de que el numero ingresado debe ser igual o mayor que 0 mi programa se ejecuto sin problemas.

EJ-145

Enunciado del algoritmo

En una ferretería, construya un algoritmo que lea el lado (real) de un cuadrado. Use la constante literal 4. Calcule el perímetro con la expresion aritmética (4 * lado) y asígnelo a perímetro. Evalúe la expresion lógica (lado > 0). Muestre por pantalla el perímetro del cuadrado. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el seno del Angulo resultado (si aplica). Verifique que las constantes estén declaradas al inicio del algoritmo.

Análisis del problema

Datos de entrada

Leer lado por teclado tipo real

Proceso

CUATRO = 4 Constante literal de tipo real

Evaluar la condición (lado > 0) para seguir con los demás procesos

Calcular el perímetro (CUATRO * lado) y asignar en perímetro como real

Calcular el seno del ángulo aplicando la función trigonométrica al perímetro

Datos de salida

Texto presentado con el resultado ("El perímetro del cuadrado es: Y")

perimetro = El resultado del cálculo del perímetro definido como real

seno_perimetro = El valor obtenido al calcular el seno del perímetro definido como real

Diagrama de flujo

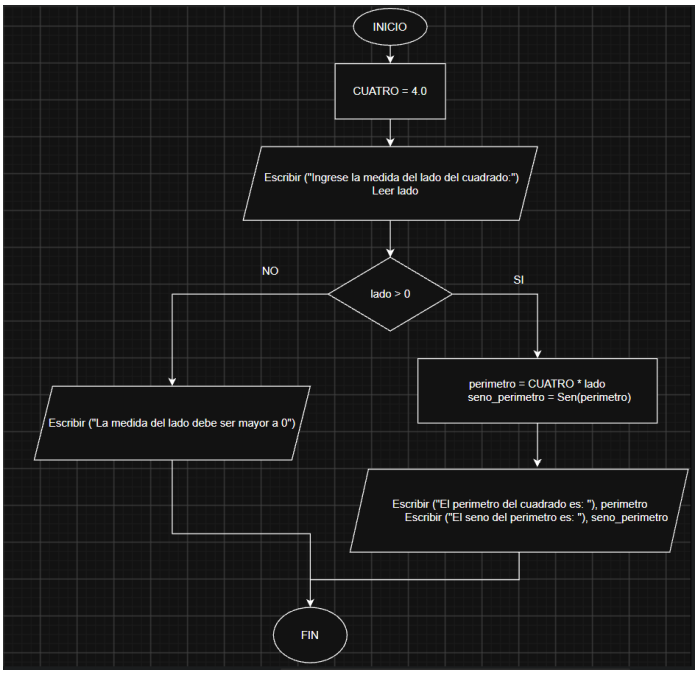


Fig. 33. Diagrama de flujo del noveno ejercicio.

Pseudocódigo

```
Algoritmo catorce
    // DEFINICION DE VARIABLES
    Definir lado Como Real
    Definir perimetro Como Real
    Definir seno_perimetro Como Real
    Definir CUATRO Como Real

    // CONSTANTE
    CUATRO = 4.0

    // LECTURA POR TECLADO
    Escribir ("Ingrese la medida del lado del cuadrado:")
    Leer lado

    // PROCESO Y VALIDACION
    Si (lado > 0) Entonces
        perimetro = CUATRO * lado
        seno_perimetro = Sen(perimetro)

        // PRESENTACION AL USUARIO
        Escribir ("El perimetro del cuadrado es: "), perimetro
        Escribir ("El seno del perimetro es: "), seno_perimetro
    Sino
        Escribir ("La medida del lado debe ser mayor a 0")
    FinSi
FinAlgoritmo
```

Fig. 34. Pseudocódigo del noveno ejercicio.

Prueba de escritorio

Paso	CUATRO	lado	perimetro	seno_perimetro	Proceso	Pantalla
1	4					
2		2			Lectura de variable lado	Ingrese la medida del lado del cuadrado:
3			8		perimetro = 4.0 * 2.0	
4				0.9894	seno_perimetro = Sen(8.0)	
5					lado > 0 -> VERDADERO	El perimetro del cuadrado es: 8 El seno del perimetro es: 0.9894

Tabla 17. Primera prueba de escritorio del noveno ejercicio.

Paso	CUATRO	lado	perimetro	seno_perimetro	Proceso	Pantalla
1	4					
2		-3			Lectura de variable lado	Ingrese la medida del lado del cuadrado:
3					lado > 0 -> FALSO	La medida del lado debe ser mayor a 0

Tabla 18. Segunda prueba de escritorio del noveno ejercicio.

Codificación en lenguaje C

CATORCE.C > main()

```c
#include <stdio.h>
//PARA CALCULAR EL SENO
#include <math.h>

int main() {
    // DEFINICION DE VARIABLES
    float lado;
    float perimetro;
    float seno_perimetro;
    const float CUATRO = 4.0;

    // LECTURA POR TECLADO
    printf("Ingrese la medida del lado del cuadrado: ");
    scanf("%f", &lado);

    // PROCESO Y VALIDACION
    if (lado > 0) {
        perimetro = CUATRO * lado;
        seno_perimetro = sin(perimetro);

        // PRESENTACION AL USUARIO
        printf("El perimetro del cuadrado es: %.2f\n", perimetro);
        printf("El seno del perimetro es: %.2f\n", seno_perimetro);
    } else {
        printf("La medida del lado debe ser mayor a 0\n");
    }

    return 0;
}
```

Fig. 35. Programación en C del noveno ejercicio.

Salida del programa

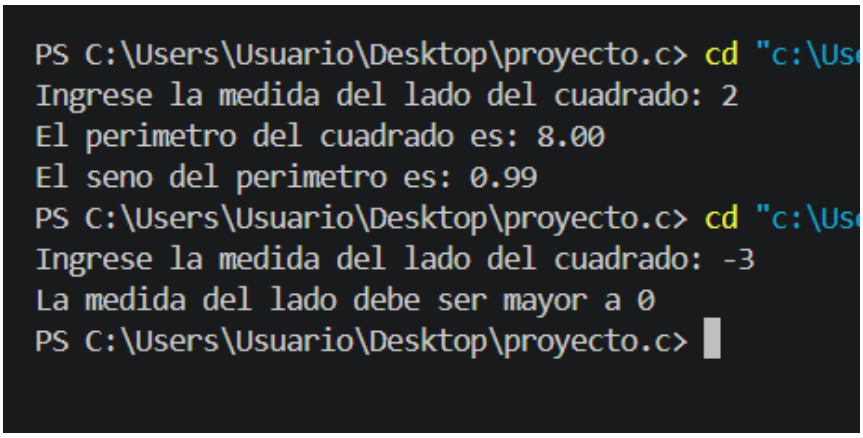


Fig. 36. Salida del programa del noveno ejercicio.

Conclusiones del algoritmo

Aprendí que se debe incluir <math.h> en C para poder usar la función trigonométrica sin() para que en el momento de ejecutar el código no me dé problemas.

Comprendí que usar el tipo Real es muy importante en cálculos geométricos y trigonométricos para conservar los decimales de los resultados y así de un valor exacto.

EJ-150

Enunciado del algoritmo

En un hotel, cree un algoritmo que lea un numero entero y por teclado. Use la constante literal 5. Evalúe la expresion lógica (y % 5 == 0 AND y != 0). Asigne el resultado booleano a es_multiplo_5. Muestre por pantalla si el número es múltiplo de 5 o no. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el resultado divido entre una constante simbólica. Verifique que las constantes estén declaradas al inicio del algoritmo.

Análisis del problema

Datos de entrada

Leer y por teclado tipo entero

Proceso

CINCO = 5 Constante literal de tipo entero

DIVISOR = 2.0 Constante simbólica de tipo real

Evaluar la expresión lógica (y % CINCO == 0 AND y != 0) y asignar el resultado booleano a es_multiplo

Calcular la división del número ingresado entre la constante simbólica (y / DIVISOR) y asignar en resultado_division como valor real

Datos de salida

es_multiplo = El resultado de la evaluación de la condición definido como lógico

resultado_division = El resultado de la división entre la constante simbólica definido como real

Diagrama de flujo

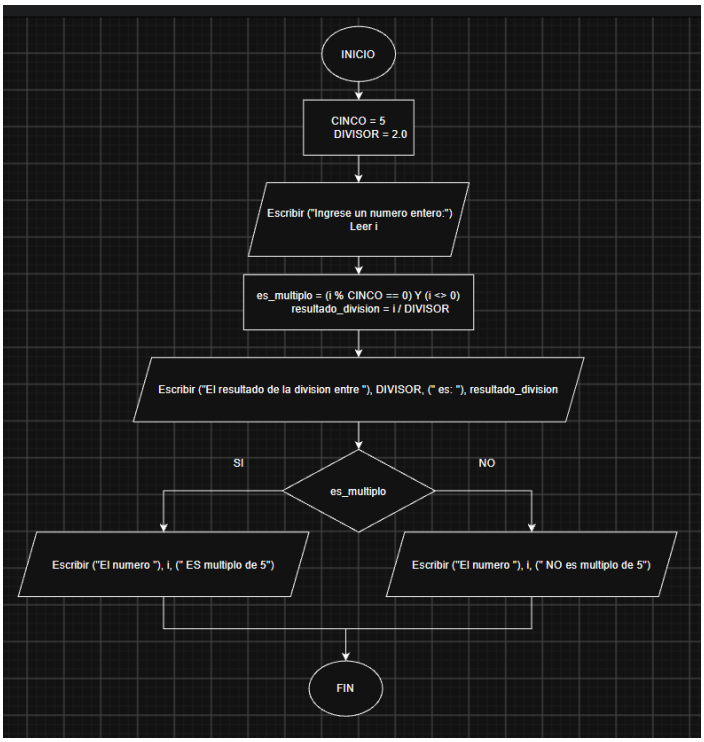


Fig. 37. Diagrama de flujo del décimo ejercicio.

Pseudocódigo

```
1  Algoritmo quince
2      // DEFINICION DE VARIABLES
3      Definir i Como Entero
4      Definir es_multiplo Como Logico
5      Definir resultado_division Como Real
6      Definir CINCO Como Entero
7      Definir DIVISOR Como Real
8
9      // CONSTANTES
10     CINCO = 5
11     DIVISOR = 2.0
12
13     // LECTURA POR TECLADO
14     Escribir ("Ingrese un numero entero:")
15     Leer i
16
17     // PROCESO
18     es_multiplo = (i % CINCO == 0) Y (i ≠ 0)
19     resultado_division = i / DIVISOR
20
21     // PRESENTACION AL USUARIO
22     Escribir ("El resultado de la division entre "), DIVISOR, (" es: "), resultado_division
23
24     Si es_multiplo Entonces
25         Escribir ("El numero "), i, (" ES multiplo de 5")
26     Sino
27         Escribir ("El numero "), i, (" NO es multiplo de 5")
28     FinSi
29 FinAlgoritmo
```

Fig. 38. Pseudocódigo del décimo ejercicio.

Prueba de escritorio

Paso	CINCO	DIVISOR	i	es_multiplo	resultado_division	Proceso	Pantalla
1	5	2					
2			15			Lectura de variable i	Ingrese un numero entero:
3				VERDADERO		(15 % 5 == 0) Y (15 != 0)	
4					7.5	resultado_division = 15 / 2.0	
5						es_multiplo -> VERDADERO	El resultado de la division entre 2 es: 7.5 El numero 15 ES multiplo de 5

Tabla 19. Primera prueba de escritorio del décimo ejercicio.

Paso	CINCO	DIVISOR	i	es_multiplo	resultado_division	Proceso	Pantalla
1	5	2					
2			12			Lectura de variable i	Ingrese un numero entero:
3				FALSO		(12 % 5 == 0) Y (12 != 0)	
4					6	resultado_division = 12 / 2.0	
5						es_multiplo -> FALSO	El resultado de la division entre 2 es: 6 El numero 12 NO es multiplo de 5

Tabla 20. Segunda prueba de escritorio del décimo ejercicio.

Codificación en lenguaje C

QUIENCE.C > main()

```c
#include <stdio.h>
// PARA EL DATO BOOLEANO
#include <stdbool.h>

int main() {
    // DEFINICION DE VARIABLES
    int y;
    bool es_multiplo;
    float resultado_division;
    const int CINCO = 5;
    const float DIVISOR = 2.0;

    // LECTURA POR TECLADO
    printf("Ingrese un numero entero: ");
    scanf("%d", &y);

    // PROCESO
    es_multiplo = (y % CINCO == 0) && (y != 0);
    resultado_division = (float)y / DIVISOR;

    // PRESENTACION AL USUARIO
    printf("El resultado de la division entre %.1f es: %.2f\n", DIVISOR, resultado_division);

    if (es_multiplo) {
        printf("El numero %d ES multiplo de 5\n", y);
    } else {
        printf("El numero %d NO es multiplo de 5\n", y);
    }

    return 0;
}
```

Fig. 39. Programación en C del décimo ejercicio.

Salida del programa



Fig. 40. Salida del programa del décimo ejercicio.

Conclusiones del algoritmo

Aprendí que se debe incluir <stdbool.h> en C para usar variables booleanas y para poder almacenar resultados de operaciones lógicas.

Comprendí que el operador % es esencial para verificar si un número es múltiplo de otro comprobando si su residuo es igual a cero gracias a eso mi código me daba las respuestas que necesitaba.

EJ-301

Enunciado del algoritmo

En una ferretería, declare tres variables enteras llamadas x, r y num1. Solicite por teclado tres números enteros (lectura) y almacénelos en esas variables. Calcule la suma, el producto y el promedio (expresión aritmética) usando una constante literal 3 como divisor. Asigne los resultados a variables suma, producto y promedio. Muestre por pantalla (escritura) un mensaje con los tres resultados. Justifique porque promedio debe ser tipo real mientras que suma y producto son enteros. Adicionalmente, calcule el cociente del resultado entre un factor de 1.5. Incluya una verificación de rango para cada variable de entrada.

Análisis del problema

Datos de entrada

Variables enteras = x, r, num1 leer por teclado

Proceso

Constante C = 3

Constante divisor A = 1.5

Límite para cada variable de mínimo 1, máximo 100

Suma de las 3 variables asignar, en suma

Multiplicación de las 3 variables asignar en producto

Promedio de las 3 variables utilizando la constante 3 asignar en promedio

Dividir cada resultado para 1.5

Datos de salida

suma = la suma de las 3 variables

producto = la multiplicación de las 3 variables

promedio = el promedio de las 3 variables

Mostrar el cociente de suma, producto y promedio dividido por 1.5

Dato extra: justificar porque promedio debe ser real mientras que suma y producto deben ser enteros.

Diagrama de flujo

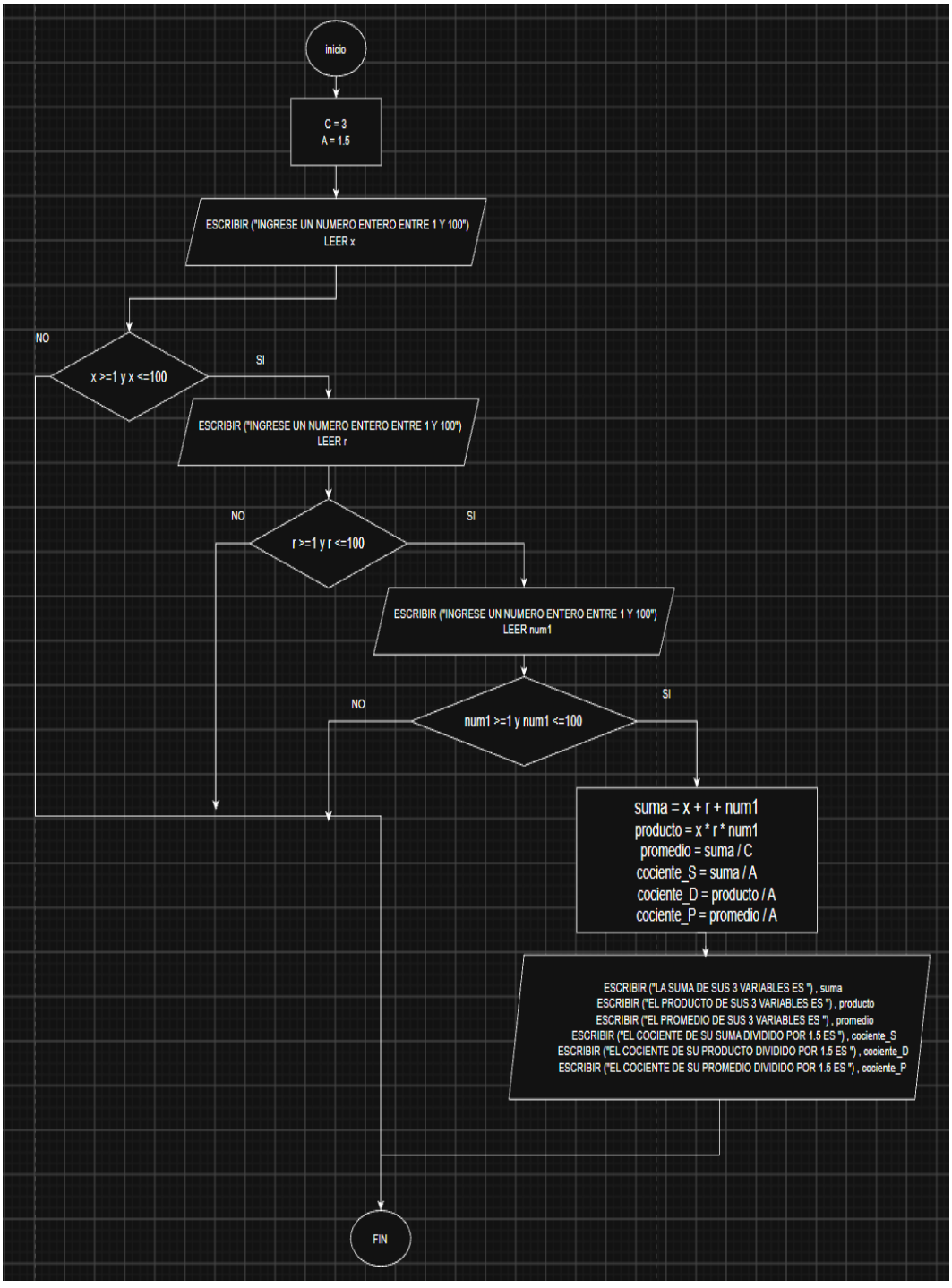


Fig. 41. Diagrama de flujo del décimo primer ejercicio.

Pseudocódigo

```
Algoritmo PRIMERO

    //DEFINIR VARIABLES
    Definir c Como Entero
    Definir A Como Real
    Definir x Como Entero
    Definir r Como Entero
    Definir num1 Como Entero
    DEFINIR suma Como Entero
    DEFINIR producto Como Entero
    DEFINIR promedio Como Real
    DEFINIR cociente_S Como Real
    DEFINIR cociente_D  Como Real
    DEFINIR cociente_P Como Real

    //DATOS CONSTANTES
    C = 3
    A = 1.5

    //LEECTURA POR TECLADO
    ESCRIBIR ("INGRESE UN NUMERO ENTERO ENTRE 1 Y 100")
    LEER x
    si x ≥1 y x ≤100 Entonces

        ESCRIBIR ("INGRESE UN NUMERO ENTERO ENTRE 1 Y 100")
        LEER r
            si r ≥1 y r ≤100 Entonces

            ESCRIBIR ("INGRESE UN NUMERO ENTERO ENTRE 1 Y 100")
            LEER num1
                si num1 ≥1 y num1 ≤100 Entonces

        //OPERACIONES DE SUMA _ PRODUCTO _ PROMEDIO
        suma = x + r + num1
        producto = x * r * num1
        promedio = suma / C
        //Cociente de suma, producto y promedio dividido por 1.5
        cociente_S = suma / A
        cociente_D = producto / A
        cociente_P = promedio / A

        //PRESENTAR LOS RESULTADOS AL USUARIO
        ESCRIBIR ("LA SUMA DE SUS 3 VARIABLES ES ") , suma
        ESCRIBIR ("EL PRODUCTO DE SUS 3 VARIABLES ES ") , producto
        ESCRIBIR ("EL PROMEDIO DE SUS 3 VARIABLES ES ") , promedio
        ESCRIBIR ("EL COCIENTE DE SU SUMA DIVIDIDO POR 1.5 ES ") , cociente_S
        ESCRIBIR ("EL COCIENTE DE SU PRODUCTO DIVIDIDO POR 1.5 ES ") , cociente_D
        ESCRIBIR ("EL COCIENTE DE SU PROMEDIO DIVIDIDO POR 1.5 ES ") , cociente_P

            sino
            Escribir ("EL TERCER NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100")
            FinSi

        sino
        Escribir ("EL SEGUNDO NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100")
        Finsi


    SiNo
    Escribir ("EL PRIMER NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100")
    FinSi


FinAlgoritmo
```

Fig. 42. Pseudocódigo del décimo primer ejercicio.

Prueba de escritorio

Paso	c	A	x	r	num1	suma	producto	promedio	cociente_S	cociente_D	cociente_P	Proceso	Pantalla
1	3	1.5											
2			10									Lectura de variable x	INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
3				5								Lectura de variable r	INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
4					15							Lectura de variable num1	INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
5						30						suma = 10 + 5 + 15	
6							750					producto = 10 * 5 * 15	
7								10				promedio = 30 / 3	
8									20			cociente_S = 30 / 1.5	
9										500		cociente_D = 750 / 1.5	
10											6.67	cociente_P = 10.0 / 1.5	
11												Impresión de resultados finales	LA SUMA DE SUS 3 VARIABLES ES 30 EL PRODUCTO DE SUS 3 VARIABLES ES 750 EL PROMEDIO DE SUS 3 VARIABLES ES 10 EL COCIENTE DE SU SUMA DIVIDIDO POR 1.5 ES 20 EL COCIENTE DE SU PRODUCTO DIVIDIDO POR 1.5 ES 500 EL COCIENTE DE SU PROMEDIO DIVIDIDO POR 1.5 ES 6.67

Tabla 21. Primera prueba de escritorio del décimo primer ejercicio.

Paso	c	A	x	r	num1	suma	producto	promedio	cociente_S	cociente_D	cociente_P	Proceso	Pantalla
1	3	1.5											
2			10									Lectura de variable x	INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
3				150								Lectura de variable r	INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
4												r >= 1 y r <= 100 -> FALSO	EL SEGUNDO NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100

Tabla 22. Segunda prueba de escritorio del décimo primer ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>

int main (){

    //DEFINIR VARIABLES
 int C , x , r , num1 , suma , producto = 0;
 double A , promedio , cociente_S , cociente_D , cociente_P = 0.0;

 //DATOS CONSTANTES
C = 3;
A = 1.5;

//LECTURA POR TECLADO + la condicion de
printf("INGRESE UN NUMERO ENTERO ENTRE 1 Y 100\n");
    scanf("%d" , &x);

if (x >= 1 && x <= 100){

    printf("INGRESE UN NUMERO ENTERO ENTRE 1 Y 100\n");
    scanf("%d" , &r);

    if (r >= 1 && r <= 100){

        printf("INGRESE UN NUMERO ENTERO ENTRE 1 Y 100\n");
        scanf("%d" , &num1);

        if (num1 >= 1 && num1 <= 100){

            //OPERACIONES DE SUMA _ PRODUCTO _ PROMEDIO
            suma = x + r + num1;
            producto = x * r * num1;
            promedio = suma / C;

            //Cociente de suma, producto y promedio dividido por 1.5
            cociente_S = suma / A;
            cociente_D = producto / A;
            cociente_P = promedio / A;

            //PRESENTAR LOS RESULTADOS AL USUARIO
            printf("LA SUMA DE SUS 3 VARIABLES ES \n%d" , suma);
            printf("\nEL PRODUCTO DE SUS 3 VARIABLES ES \n%d" , producto);
            printf("\nEL PROMEDIO DE SUS 3 VARIABLES ES \n%.2f" , promedio);
            printf("\nEL COCIENTE DE SU SUMA DIVIDIDO POR 1.5 ES \n%.2f" , cociente_S);
            printf("\nEL COCIENTE DE SU PRODUCTO DIVIDIDO POR 1.5 ES \n%.2f" ,
cociente_D);
            printf("\nEL COCIENTE DE SU PROMEDIO DIVIDIDO POR 1.5 ES \n%.2f" ,
cociente_P);

        } else {
            printf ("EL TERCER NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100");
        }

    } else {
        printf ("EL SEGUNDO NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100");
    }

} else {
    printf ("EL PRIMER NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100");
}

return 0;
}
```

Fig. 43. Programación en C del décimo primer ejercicio.

Salida del programa

```
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuario\Des
INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
500
EL PRIMER NUMERO NO ESTA EN EL RANGO INDICADO ENTRE 1 AL 100
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuario\Des
INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
10
INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
5
INGRESE UN NUMERO ENTERO ENTRE 1 Y 100
15
LA SUMA DE SUS 3 VARIABLES ES
30
EL PRODUCTO DE SUS 3 VARIABLES ES
750
EL PROMEDIO DE SUS 3 VARIABLES ES
10.00
EL COCIENTE DE SU SUMA DIVIDIDO POR 1.5 ES
20.00
EL COCIENTE DE SU PRODUCTO DIVIDIDO POR 1.5 ES
500.00
EL COCIENTE DE SU PROMEDIO DIVIDIDO POR 1.5 ES
6.67
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 44. Salida del programa del décimo primer ejercicio.

Conclusiones del algoritmo

Al calcular el promedio dividiendo para 3, el resultado no siempre va a dar un número entero, sino que la mayoría de las veces genera decimales como por ejemplo 10 / 3 = 3.33. Por esta razón, la variable promedio se debe declarar como tipo de dato real (double o float), a diferencia de la suma y el producto, que al trabajar solo con enteros siempre darán como resultado un número entero.

Con el uso del if y el else aprendí que al poner if esa condición debe cumplirse, en cambio con el else se ejecuta otra acción si la condición del if es incorrecta

EJ-306

Enunciado del algoritmo

En un hotel, construya un algoritmo que lea el nombre de un estudiante (cadena) y tres notas (reales entre 0 y 10). Use una constante simbólica NOTA_MIN = 7.0. Calcule el promedio con la expresión aritmética ((nota1 + nota2 + nota3) / 3) y asígnelo a promedio. Evalúe la expresión relacional (promedio >= NOTA_MIN). Muestre por pantalla el nombre del estudiante, su promedio y si aprobó o reprobó. Adicionalmente, calcule el inverso del resultado si este no es cero. Incluya una verificación de rango para cada variable de entrada.

Análisis del problema

Datos de entrada

Variable tipo cadena de carácter estudiante, leer por teclado

Variables tipo real nota1, nota2, nota3 leer por teclado

Procesos

Constante NOTA_MIN = 7.0

Constante DIVI = 3

Constante UNO = 1

Límite para cada variable de tipo real del 0 al 10

Aplicar la condición del límite para cada variable

Calcular el promedio sumando las 3 notas y dividiéndola para 3 y eso asignar promedio

Evaluar el promedio y si este es mayor o igual a 7 aprueba si no desaprueba asignarla en verificación

Calcular el inverso del promedio poniendo como numerador a (UNO) y denominador a promedio, pero depende si el promedio no es 0

Datos de salida

estudiante = el nombre del estudiante

promedio= la suma de las 3 notas divididas para 3

verificacion = aprueba, desaprueba

inverso = resultado de dividir UNO para el promedio

Diagrama de flujo

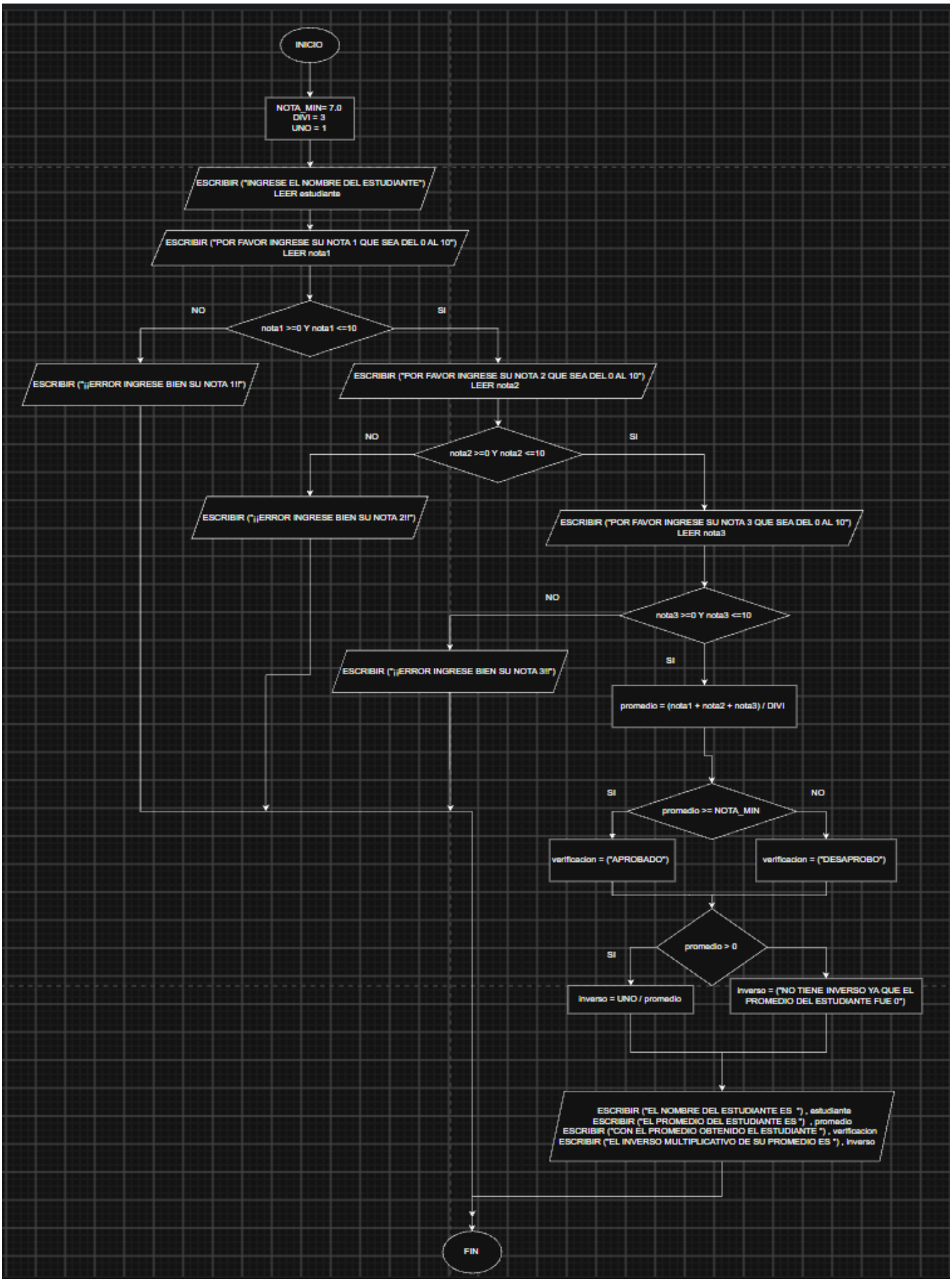


Fig. 45. Diagrama de flujo del décimo segundo ejercicio.

Pseudocódigo

```
Algoritmo PromedioEstudiante
    // Definición de variables constantes iniciales
    NOTA_MIN = 7.0
    DIVI = 3
    UNO = 1
    //Ingreso de variables por el usuario
    Escribir "INGRESE EL NOMBRE DEL ESTUDIANTE"
    Leer estudiante

    Escribir "POR FAVOR INGRESE SU NOTA 1 QUE SEA DEL 0 AL 10"
    Leer nota1

    Si nota1 ≥ 0 Y nota1 ≤ 10 Entonces
        Escribir "POR FAVOR INGRESE SU NOTA 2 QUE SEA DEL 0 AL 10"
        Leer nota2

        Si nota2 ≥ 0 Y nota2 ≤ 10 Entonces
            Escribir "POR FAVOR INGRESE SU NOTA 3 QUE SEA DEL 0 AL 10"
            Leer nota3

            Si nota3 ≥ 0 Y nota3 ≤ 10 Entonces
                // Cálculo del promedio
                promedio = (nota1 + nota2 + nota3) / DIVI

                // Verificación de aprobación
                Si promedio ≥ NOTA_MIN Entonces
                    verificacion = "APROBADO"
                Sino
                    verificacion = "DESAPROBO"
                FinSi

                // Cálculo del inverso multiplicativo
                Si promedio > 0 Entonces
                    inverso_num = UNO / promedio
                    Escribir "EL INVERSO MULTIPLICATIVO DE SU PROMEDIO ES ", inverso_num
                Sino
                    Escribir "NO TIENE INVERSO YA QUE EL PROMEDIO DEL ESTUDIANTE FUE 0"
                FinSi

                // Impresión de resultados principales
                Escribir "EL NOMBRE DEL ESTUDIANTE ES ", estudiante
                Escribir "EL PROMEDIO DEL ESTUDIANTE ES ", promedio
                Escribir "CON EL PROMEDIO OBTENIDO EL ESTUDIANTE ", verificacion

            Sino
                Escribir "ERROR INGRESE BIEN SU NOTA 3!"
            FinSi

        Sino
            Escribir "ERROR INGRESE BIEN SU NOTA 2!"
        FinSi

    Sino
        Escribir "ERROR INGRESE BIEN SU NOTA 1!"
    FinSi
FinAlgoritmo
```

Fig. 46. Pseudocódigo del décimo segundo ejercicio.

Prueba de escritorio

Paso	NOTA_MIN	DIVI	UNO	estudiante	nota1	nota2	nota3	promedio	verificacion	inverso_num	Proceso	Pantalla
1	7	3	1									
2				CARLOS							Lectura de variable estudiante	INGRESE EL NOMBRE DEL ESTUDIANTE
3					8						Lectura de variable nota1	POR FAVOR INGRESE SU NOTA 1 QUE SEA DEL 0 AL 10
4						9					Lectura de variable nota2	POR FAVOR INGRESE SU NOTA 2 QUE SEA DEL 0 AL 10
5							7				Lectura de variable nota3	POR FAVOR INGRESE SU NOTA 3 QUE SEA DEL 0 AL 10
6								8			promedio = (8.0 + 9.0 + 7.0) / 3	
7									APROBADO		promedio >= NOTA_MIN -> VERDADERO	
8										0.125	inverso_num = 1 / 8.0	EL INVERSO MULTIPLICATIVO DE SU PROMEDIO ES 0.125
9											Impresión de resultados finales	EL NOMBRE DEL ESTUDIANTE ES CARLOS EL PROMEDIO DEL ESTUDIANTE ES 8 CON EL PROMEDIO OBTENIDO EL ESTUDIANTE APROBADO

Tabla 23. Primera prueba de escritorio del décimo segundo ejercicio.

Paso	NOTA_MIN	DIVI	UNO	estudiante	nota1	nota2	nota3	promedio	verificacion	inverso_num	Proceso	Pantalla
1	7	3	1									
2				MARIA							Lectura de variable estudiante	INGRESE EL NOMBRE DEL ESTUDIANTE
3					5						Lectura de variable nota1	POR FAVOR INGRESE SU NOTA 1 QUE SEA DEL 0 AL 10
4						6					Lectura de variable nota2	POR FAVOR INGRESE SU NOTA 2 QUE SEA DEL 0 AL 10
5							4				Lectura de variable nota3	POR FAVOR INGRESE SU NOTA 3 QUE SEA DEL 0 AL 10
6								5			promedio = (5.0 + 6.0 + 4.0) / 3	
7									DESAPROBO		promedio >= NOTA_MIN -> FALSO	
8										0.2	inverso_num = 1 / 5.0	EL INVERSO MULTIPLICATIVO DE SU PROMEDIO ES 0.2
9											Impresión de resultados finales	EL NOMBRE DEL ESTUDIANTE ES MARIA EL PROMEDIO DEL ESTUDIANTE ES 5 CON EL PROMEDIO OBTENIDO EL ESTUDIANTE DESAPROBO

Tabla 24. Segunda prueba de escritorio del décimo segundo ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    // DECLARACIÓN DE VARIABLES
    char *estudiante;
    const char* verificacion;
    int DIVI, UNO;
    float nota1, nota2, nota3, promedio, NOTA_MIN, inverso;

    // ASIGNACIÓN DE CONSTANTES
    NOTA_MIN = 7.0;
    DIVI = 3;
    UNO = 1;

    // INGRESO DEL NOMBRE DEL ESTUDIANTE
    printf("INGRESE EL NOMBRE DEL ESTUDIANTE\n");
    scanf("%s", estudiante);

    // INGRESO Y VALIDACIÓN DE LA NOTA 1
    printf("POR FAVOR INGRESE SU NOTA 1 QUE SEA DEL 0 AL 10\n");
    scanf("%f", &nota1);

    if (nota1 >= 0 && nota1 <= 10) {

        // INGRESO Y VALIDACIÓN DE LA NOTA 2
        printf("POR FAVOR INGRESE SU NOTA 2 QUE SEA DEL 0 AL 10\n");
        scanf("%f", &nota2);

        if (nota2 >= 0 && nota2 <= 10) {

            // INGRESO Y VALIDACIÓN DE LA NOTA 3
            printf("POR FAVOR INGRESE SU NOTA 3 QUE SEA DEL 0 AL 10\n");
            scanf("%f", &nota3);

            if (nota3 >= 0 && nota3 <= 10) {

                // CÁLCULO DEL PROMEDIO
                promedio = (nota1 + nota2 + nota3) / DIVI;

                // VERIFICACIÓN DE APROBACIÓN
                if (promedio >= NOTA_MIN) {
                    verificacion = "APROBADO";
                } else {
                    verificacion = "DESAPROBO";
                }

                // CÁLCULO DEL INVERSO MULTIPLICATIVO
                if (promedio > 0) {
                    inverso = UNO / promedio;
                    printf("EL INVERSO MULTIPLICATIVO DE SU PROMEDIO ES %.2f\n",
inverso);
                } else {
                    printf("NO TIENE INVERSO YA QUE EL PROMEDIO DEL ESTUDIANTE FUE 0\n"
);
                }

                // IMPRESIÓN DE RESULTADOS
                printf("EL NOMBRE DEL ESTUDIANTE ES %s\n", estudiante);
                printf("EL PROMEDIO DEL ESTUDIANTE ES %.2f\n", promedio);
                printf("CON EL PROMEDIO OBTENIDO EL ESTUDIANTE %s\n", verificacion);

            } else {
                printf(" !!ERROR INGRESE BIEN SU NOTA 3!!\n");
            }

        } else {
            printf(" !!ERROR INGRESE BIEN SU NOTA 2!!\n");
        }

    } else {
        printf(" !!ERROR INGRESE BIEN SU NOTA 1!!\n");
    }

    return 0;
}
```

Fig. 47. Programación en C del décimo segundo ejercicio.

Salida del programa

```
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\U
INGRESE EL NOMBRE DEL ESTUDIANTE
Jinmy
POR FAVOR INGRESE SU NOTA 1 QUE SEA DEL 0 AL 10
11
 !!ERROR INGRESE BIEN SU NOTA 1!!
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\U
INGRESE EL NOMBRE DEL ESTUDIANTE
Javier
POR FAVOR INGRESE SU NOTA 1 QUE SEA DEL 0 AL 10
8
POR FAVOR INGRESE SU NOTA 2 QUE SEA DEL 0 AL 10
9
POR FAVOR INGRESE SU NOTA 3 QUE SEA DEL 0 AL 10
7
EL INVERSO MULTIPLICATIVO DE SU PROMEDIO ES 0.12
EL NOMBRE DEL ESTUDIANTE ES Javier
EL PROMEDIO DEL ESTUDIANTE ES 8.00
CON EL PROMEDIO OBTENIDO EL ESTUDIANTE APROBADO
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 48. Salida del programa del décimo segundo ejercicio.

Conclusiones del algoritmo

En C hay que tener mucho cuidado con la sintaxis al pedir datos. Si dentro del scanf se pone la precisión %.2f en lugar de solo "%f", el programa no lee bien y nos da errores de compilación, pero como C eso no marca como error entonces no me di cuenta.

El uso de constantes como NOTA_MIN permite evaluar fácilmente si el estudiante aprueba o no según su promedio. Además, verificar con un if que el promedio sea mayor a cero antes de calcular su inverso multiplicativo es fundamental para evitar que el programa intente dividir para cero y termine fallando por un error matemático.

EJ-314

Enunciado del algoritmo

En una gasolinera, construya un algoritmo que lea un carácter b por teclado. Use la constante literal 32. Evalúe la expresión lógica (b >= 'A' AND b <= 'Z'). Si es verdadero, calcule el carácter en minúscula con la expresión aritmética (carácter + 32) y asígnelo a s. Muestre por pantalla el carácter original y el convertido. Justifique el tipo de dato de cada variable. Adicionalmente, reste un descuento del 5% al resultado final. Use identificadores descriptivos para todas las variables.

Análisis del problema

Datos de entrada

Variable tipo carácter b leer por teclado (es un carácter porque es una letra)

Proceso

Constante SUMATIVO = 32 definir como entero

Constante DESCUENTO = 0.05 definir como real

Evaluar la expresión lógica donde b debe ser mayor o igual a 'A' pero b debe ser menor o igual a 'Z' (utilizando el código ASCII) si la expresión calculada es verdadera se volverla minúscula utilizando la expresión (carácter + 32) o si es falsa el resultado será lo mismo que escribió el usuario y esto se asigna en s

A la cantidad final ósea al número de la letra en minúscula en el código ASCII se le resta el 5%

Datos de salida

b = la letra ingresada por el usuario

s = la letra convertida en minúscula

porcentaje = el numero de la letra en el código ASCII restado el 5%

Diagrama de flujo

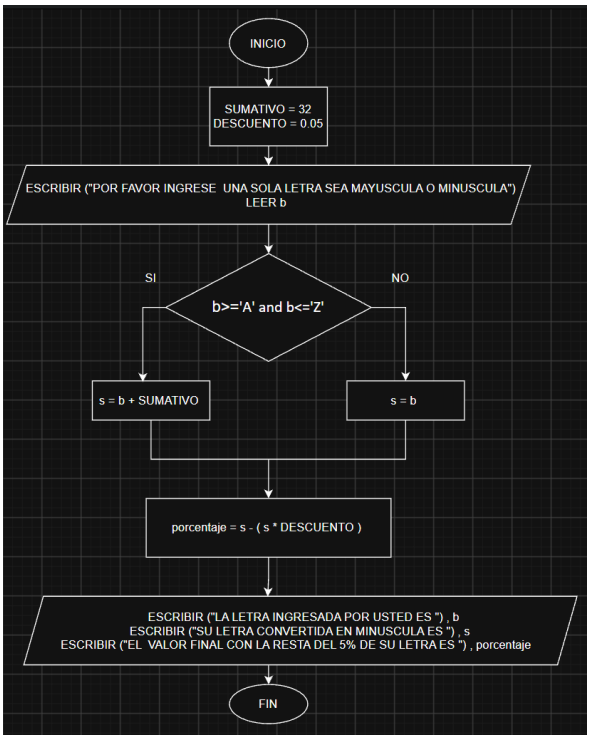


Fig. 49. Diagrama de flujo del décimo tercer ejercicio.

Pseudocódigo

```
Algoritmo TRESSS
    //Definir VARIABLES
    Definir SUMATIVO Como Entero
    Definir DESCUENTO Como Real
    Definir b Como Caracter
    Definir s Como CARACTER
    Definir porcentaje Como Real

    //CONSTANTES
    SUMATIVO = 32
    DESCUENTO = 0.05

    //INGRESO DE VARIABLES POR EL USUARIO
    ESCRIBIR ("POR FAVOR INGRESE UNA SOLA LETRA SEA MAYUSCULA O MINUSCULA")
    LEER b

    //Proceso
    si b≥'A' y b≤'Z'Entonces
        s = Minusculas(b)
        sino s = b
    FinSi

    //LA APLICACION DEL DESCUENTO NO SE LE PUEDE APLICAR
    //YA QUE ELPSEUDOCODIGO NO TIENE LIBRERIA DE CODIGO ASCII
    //porcentaje = s - ( s * DESCUENTO )

    //PRSENTAR AL USUARIO
    ESCRIBIR ("LA LETRA INGRESADA POR USTED ES ") , b
    ESCRIBIR ("SU LETRA CONVERTIDA EN MINUSCULA ES ") , s
    ESCRIBIR ("EL VALOR DE LA RESTA DEL 5% NO SE PUDO EJECUTAR YA QUE EN EL PSEUDOCODIGO NO PUEDE TRASFORMAR UNA LETRA A UN NUMERO")

FinAlgoritmo
```

Fig. 50. Pseudocódigo del décimo tercer ejercicio.

Prueba de escritorio

Paso	SUMATIVO	DESCUENTO	b	s	porcentaje	Proceso	Pantalla
1	32	0.05					
2			A			Lectura de variable b	POR FAVOR INGRESE UNA SOLA LETRA SEA MAYUSCULA O MINUSCULA
3				a		b >= 'A' y b <= 'Z' -> s = 'a' (ASCII 97)	
4					92.15	porcentaje = 97 - (97 * 0.05)	
5							LA LETRA INGRESADA POR USTED ES A SU LETRA CONVERTIDA EN MINUSCULA ES a EL VALOR DE LA RESTA DEL 5% ES 92.15

Tabla 25. Primera prueba de escritorio del décimo tercer ejercicio.

Paso	SUMATIVO	DESCUENTO	b	s	porcentaje	Proceso	Pantalla
1	32	0.05					
2			b			Lectura de variable b	POR FAVOR INGRESE UNA SOLA LETRA SEA MAYUSCULA O MINUSCULA
3				b		b >= 'A' y b <= 'Z' -> FALSO -> s = 'b' (ASCII 98)	
4					93.1	porcentaje = 98 - (98 * 0.05)	
5							LA LETRA INGRESADA POR USTED ES b SU LETRA CONVERTIDA EN MINUSCULA ES b EL VALOR DE LA RESTA DEL 5% ES 93.1

Tabla 26. Segunda prueba de escritorio del décimo tercer ejercicio.

Codificación en lenguaje C

TRESSS.c

```c
#include <stdio.h>

int main (){

//DECLARACION DE VARIABLES
float pocentaje = 0.0;
char b;
char s;

//CONSTANTES
int SUMATIVO = 32;
float DESCUENTO = 0.05;

//INGRESO DE VARIABLES POR EL USUARIO
printf("POR FAVOR INGRESE UNA SOLA LETRA SEA MAYUSCULA O MINUSCULA\n");
    scanf("%c" , &b);

//Proceso
if (b>='A' && b<='z'){
    s = b + 32;
}else {
    s = b;
}

//CALCULO DEL PORCENTAJE
pocentaje = s - (s * DESCUENTO);

//PRESENTAR AL USUARIO
printf ("LA LETRA INGRESADA POR USTED ES %c \n" , b);
printf ("SU LETRA CONVERTIDA EN MINUSCULA ES %c\n" , s  );
printf ("EL VALOR DE LA RESTA DEL 5%% %.2f\n" , pocentaje );

}
```

Fig. 51. Programación en C del décimo tercer ejercicio.

Salida del programa

```
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuari
POR FAVOR INGRESE UNA SOLA LETRA SEA MAYUSCULA O MINUSCULA
A
LA LETRA INGRESADA POR USTED ES A
SU LETRA CONVERTIDA EN MINUSCULA ES a
EL VALOR DE LA RESTA DEL 5% 92.15
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuari
POR FAVOR INGRESE UNA SOLA LETRA SEA MAYUSCULA O MINUSCULA
a
LA LETRA INGRESADA POR USTED ES a
SU LETRA CONVERTIDA EN MINUSCULA ES a
EL VALOR DE LA RESTA DEL 5% 92.15
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 52. Salida del programa del décimo tercer ejercicio.

Conclusiones del algoritmo

Comprendí que en la programación en C es obligatorio indicarle al sistema el tipo de dato exacto que se va a leer. Me quedó claro qué %c se usa únicamente para pedir una sola letra y qué %s sirve para leer una palabra o cadena de texto completa.

En este ejercicio pude ver la diferencia clave entre PSeInt y C en PSeInt trata las letras puramente como texto, mientras que en C cada letra tiene un valor numérico equivalente en código ASCII. Gracias a esto, en C pude sumarle 32 a la variable para cambiar la letra a minúscula y luego restarle el 5% al valor final sin complicación.

EJ-318

Enunciado del algoritmo

En una tienda de electrónica, diseñe un algoritmo que lea el peso (real, en kg) y la altura (real, en metros). Calcule el IMC con la expresión aritmética (peso / (altura * altura)) y asígnelo a imc. Use constantes simbólicas LIMITE_BAJO = 18.5 y LIMITE_ALTO = 25.0. Evalúe la expresión lógica (imc >= LIMITE_BAJO AND imc <= LIMITE_ALTO). Muestre por pantalla el IMC calculado y si está en rango normal. Adicionalmente, determine si el resultado es mayor que un umbral predefinido. Implemente una bandera booleana para indicar si el resultado es válido.

Análisis del problema

Datos de entrada

Leer peso con su prefijo Kg, definido como un número real.

Leer altura con su prefijo m, definida como un número real.

Proceso

Constante LIMITE_BAJO = 18.5, definida como un número real.

Constante LIMITE_ALTO = 25.0, definida como un número real.

Constante UMBRAL = 25.0, definida como un número real (el ejercicio pide un umbral predefinido, pero no lo especifica, por lo que se toma el valor de LIMITE_ALTO).

Mediante una bandera booleana es_valido, comprobar si los datos ingresados por el usuario son mayores a 0 (peso > 0 Y altura > 0). Y si es mayor a 0 ejecutar el programa.

Calcular el IMC con la expresión (peso / (altura * altura)) y asignarlo a imc.

Evaluar la expresión lógica (imc >= LIMITE_BAJO Y imc <= LIMITE_ALTO) para comprobar que esté en el rango normal y asignar el resultado a en_rango.

Determinar si (imc > UMBRAL) y asignar el resultado a es_mayor. Si es verdadero, presentar que el IMC es mayor que el umbral; si es falso, presentar que no es mayor (es menor o igual).

Datos de salida

es_valido = es falso, mostrar “Datos inválidos: peso y altura deben ser mayores que 0”.

imc = resultado de la expresión aritmética calculada (solo si es_valido es verdadero).

en_rango = indica si el IMC está dentro de los límites (normal) o fuera de ellos.

es_mayor = resultado de (imc > UMBRAL): muestra si el IMC es mayor o no mayor que el umbral.

Diagrama de flujo

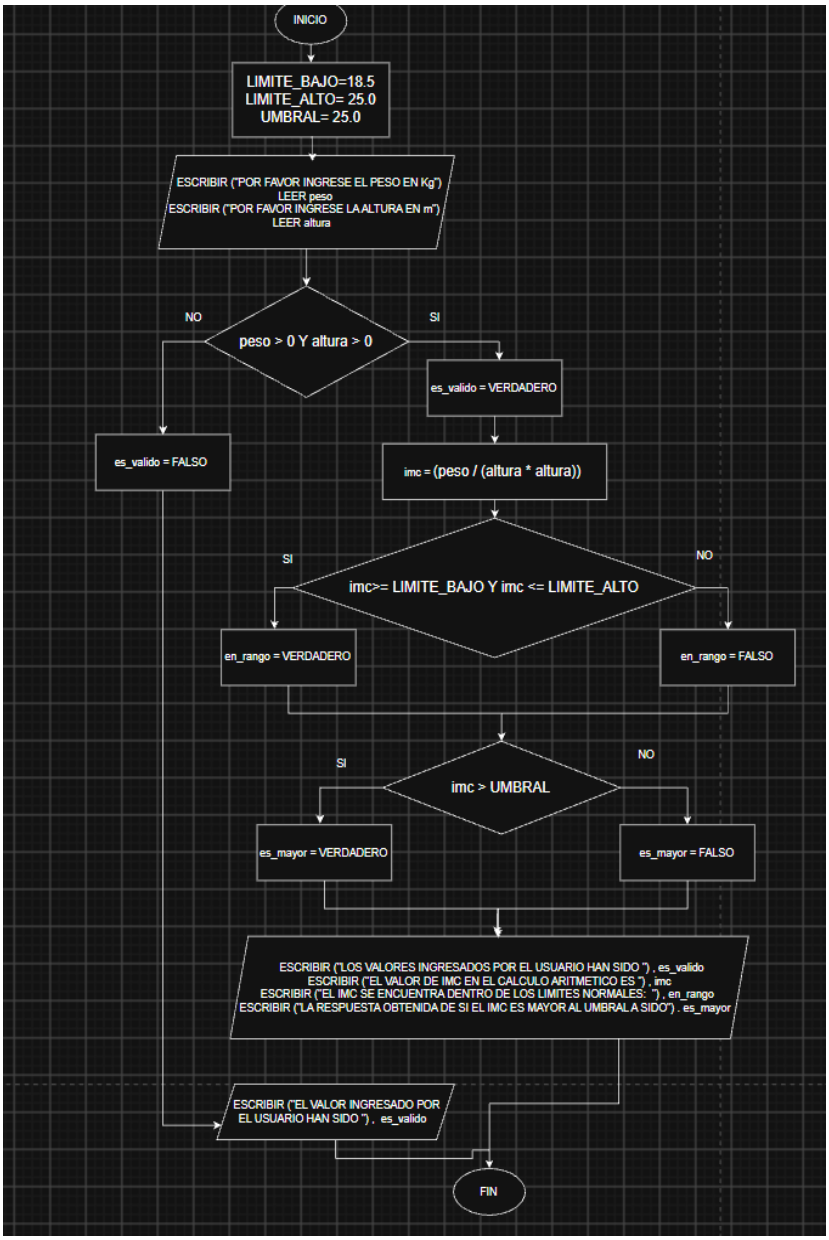


Fig. 53. Diagrama de flujo del décimo cuarto ejercicio.

Pseudocódigo

```
Algoritmo cuatro

    //Definir VARIABLES
    Definir peso , altura Como Real
    Definir LIMITE_BAJO , LIMITE_ALTO , UMBRAL ,imc Como Real
    Definir es_mayor , en_rango ,es_valido Como Logico

    //Leer por teclado
    ESCRIBIR ("POR FAVOR INGRESE EL PESO EN Kg")
    LEER peso
    ESCRIBIR ("POR FAVOR INGRESE LA ALTURA EN m")
    LEER altura
    //COSTANTES
    LIMITE_BAJO=18.5
    LIMITE_ALTO= 25.0
    UMBRAL= 25.0

    //DECLARAMOS LA BANDERA BOOLEANA
    si (peso > 0 Y altura > 0) Entonces
        es_valido = Verdadero

        //Proceso de resolucion
        imc = (peso / (altura * altura) )

        si imc≥ LIMITE_BAJO Y imc ≤ LIMITE_ALTO Entonces
            en_rango = VERDADERO
            sino en_rango = FALSO
        FinSi

        si imc > UMBRAL Entonces
            es_mayor = VERDADERO
            sino es_mayor = FALSO
        FinSi

        //PRESENTAR POR PANTALLA
        ESCRIBIR ("LOS VALORES INGRESADOS POR EL USUARIO HAN SIDO ") , es_valido
        ESCRIBIR ("EL VALOR DE IMC EN EL CALCULO ARITMETICO ES ") , imc
        ESCRIBIR ("EL IMC SE ENCUENTRA DENTRO DE LOS LIMITES NORMALES: ") , en_rango
        ESCRIBIR ("LA RESPUESTA OBTENIDA DE SI EL IMC ES MAYOR AL UMBRAL A SIDO ") , es_mayor


    SINO
        //CONDICION PARA QUE EL EJERCICIO NO SE RESUELVA
        ESCRIBIR ("EL VALOR INGRESADO POR EL USUARIO HAN SIDO ") , es_valido
    FinSi

FinAlgoritmo
```

Fig. 54. Pseudocódigo del décimo cuarto ejercicio.

Prueba de escritorio

Paso	LIMITE_BAJO	LIMITE_ALTO	UMBRAL	peso	altura	es_valido	imc	en_rango	es_mayor	Proceso	Pantalla
1	18.5	25	25								
2				70						Lectura de variable peso	POR FAVOR INGRESE EL PESO EN Kg
3					1.75					Lectura de variable altura	POR FAVOR INGRESE LA ALTURA EN m
4						VERDADERO				peso > 0 Y altura > 0	
5							22.86			imc = 70.0 / (1.75 * 1.75)	
6								VERDADER		imc >= LIMITE_BAJO Y imc <= LIMITE_ALTO	
7									FALSO	imc > UMBRAL	
8											LOS VALORES INGRESADOS POR EL USUARIO HAN SIDO VERDADERO EL VALOR DE IMC EN EL CALCULO ARITMETICO ES 22.86 EL IMC SE ENCUENTRA DENTRO DE LOS LIMITES NORMALES: VERDADERO LA RESPUESTA OBTENIDA DE SI EL IMC ES MAYOR AL UMBRAL A SIDO FALSO

Tabla 27. Primera prueba de escritorio del décimo cuarto ejercicio.

Paso	LIMITE_BAJO	LIMITE_ALTO	UMBRAL	peso	altura	es_valido	imc	en_rango	es_mayor	Proceso	Pantalla
1	18.5	25	25								
2				85						Lectura de variable peso	POR FAVOR INGRESE EL PESO EN Kg
3					1.7					Lectura de variable altura	POR FAVOR INGRESE LA ALTURA EN m
4						VERDADERO				peso > 0 Y altura > 0	
5							29.41			imc = 85.0 / (1.70 * 1.70)	
6								FALSO		imc >= LIMITE_BAJO Y imc <= LIMITE_ALTO	
7									VERDADERO	imc > UMBRAL	
8											LOS VALORES INGRESADOS POR EL USUARIO HAN SIDO VERDADERO EL VALOR DE IMC EN EL CALCULO ARITMETICO ES 29.41 EL IMC SE ENCUENTRA DENTRO DE LOS LIMITES NORMALES: FALSO LA RESPUESTA OBTENIDA DE SI EL IMC ES MAYOR AL UMBRAL A SIDO VERDADERO

Tabla 28. Segunda prueba de escritorio del décimo cuarto ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>
// LIBRERIA PARA DECLARAR BOOLEANOS
#include <stdbool.h>

int main() {

    // DEFINICION DE VARIABLES
    float peso, altura;
    float LIMITE_BAJO, LIMITE_ALTO, UMBRAL, imc;
    bool es_mayor, en_rango, es_valido;

    // LEER POR TECLADO
    printf("POR FAVOR INGRESE EL PESO EN Kg\n");
    scanf("%f", &peso);
    printf("POR FAVOR INGRESE LA ALTURA EN m\n");
    scanf("%f", &altura);

    // CONSTANTES
    LIMITE_BAJO = 18.5;
    LIMITE_ALTO = 25.0;
    UMBRAL = 25.0;

    // DECLARAR LA BANDERA BOOLEANA
    if (peso > 0 && altura > 0) {

        es_valido = true;

        // proceso de resolucion
        imc = (peso / (altura * altura));

        if (imc >= LIMITE_BAJO && imc <= LIMITE_ALTO) {
            en_rango = true;
        } else {
            en_rango = false;
        }

        if (imc > UMBRAL) {
            es_mayor = true;
        } else {
            es_mayor = false;
        }

        // PRESENTAR PANTALLA
        printf("LOS VALORES INGRESADOS POR EL USUARIO HAN SIDO %s\n", es_valido ? "Verdadero " : "Falso ");
        printf("EL VALOR DE IMC EN EL CALCULO ARITMETICO ES %.2f \n", imc);
        printf("EL IMC SE ENCUENTRA DENTRO DE LOS LIMITES NORMALES: %s \n", en_rango ? "Verdadero " : "Falso ");
        printf("LA RESPUESTA OBTENIDA DE SI EL IMC ES MAYOR AL UMBRAL A SIDO %s \n", es_mayor ? "Verdadero " : "Falso ");

    } else {
        printf("EL VALOR INGRESADO POR EL USUARIIO HAN SIDO %s \n", es_mayor ? "Verdadero " : "Falso ");
    }

    return 0;
}
```

Fig. 55. Programación en C del décimo cuarto ejercicio.

Salida del programa

```
Problemas  Salida  Consola de depuración  Terminal  Puertos

PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuario\Desktop\pr
POR FAVOR INGRESE EL PESO EN Kg
0
POR FAVOR INGRESE LA ALTURA EN m
0
EL VALOR INGRESADO POR EL USUARIIO HAN SIDO Falso
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuario\Desktop\pr
POR FAVOR INGRESE EL PESO EN Kg
85
POR FAVOR INGRESE LA ALTURA EN m
1.7
LOS VALORES INGRESADOS POR EL USUARIO HAN SIDO Verdadero
EL VALOR DE IMC EN EL CALCULO ARITMETICO ES 29.41
EL IMC SE ENCUENTRA DENTRO DE LOS LIMITES NORMALES: Falso
LA RESPUESTA OBTENIDA DE SI EL IMC ES MAYOR AL UMBRAL A SIDO Verdadero
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 56. Salida del programa del décimo cuarto ejercicio.

Conclusiones del algoritmo

Aprendí que una bandera booleana (como es_valido) actúa como un mecanismo de seguridad o llave que valida las condiciones iniciales antes de procesar la información. Si la condición se cumple ósea los datos son coherentes, como un peso y altura mayores a cero, el programa ejecuta todo el cálculo e impresiones con normalidad, de lo contrario, detiene el flujo principal y muestra un mensaje de error advirtiendo que los valores no son válidos.

Comprendí que en lenguaje C los valores booleanos numéricamente representan solo 1 (verdadero) o 0 (falso) cuando se imprimen directamente con %d. Pero para mostrar un texto al usuario como "Verdadero" o "Falso", se utiliza el operador ternario (condición ? "Verdadero" : "Falso") junto con el formato de texto %s. Esto permite evaluar la condición y devolver la palabra deseada según corresponda el resultado.

EJ-334

Enunciado del algoritmo

En un gimnasio, cree un algoritmo que lea un año (entero) por teclado. Use constantes literales 4, 100 y 400. Evalúe la expresión logica compuesta ((año % 4 == 0) AND (año % 100 != 0 OR año % 400 == 0)). Asigne el resultado booleano a es_bisiesto. Muestre por pantalla si el año es o no bisiesto. Justifique el uso de los operadores AND y OR en la expresión. Adicionalmente, calcule la media aritmética de varios resultados. Use identificadores descriptivos para todas las variables.

Análisis del problema

Datos de entrada

Leer año por teclado definir como entero

Leer numero1, numero2, numero3 por teclado definir como entero

Proceso

Constante CUATRO = 4 definir como entero

Constante CIEN = 100 definir como entero

Constante CUATROCIENTOS = 400 definir como entero

Evaluar la expresión lógica ((año % 4 == 0) AND (año % 100 != 0 OR año % 400 == 0)) con un booleano es_bisiesto para ver si el año es bisiesto o no es bisiesto, justificar por que se utiliza AND y OR en la expresión

Realizar la media aritmética de numero1, numero2, numero3 asignar en media utilice media $= (numero1 + numero2 + numero3)/3$

Datos de salida

es_bisiesto = verdad o falso depende del resultado de la expresion

media = la respuesta de la formula de la media aritmética definir como real

Diagrama de flujo

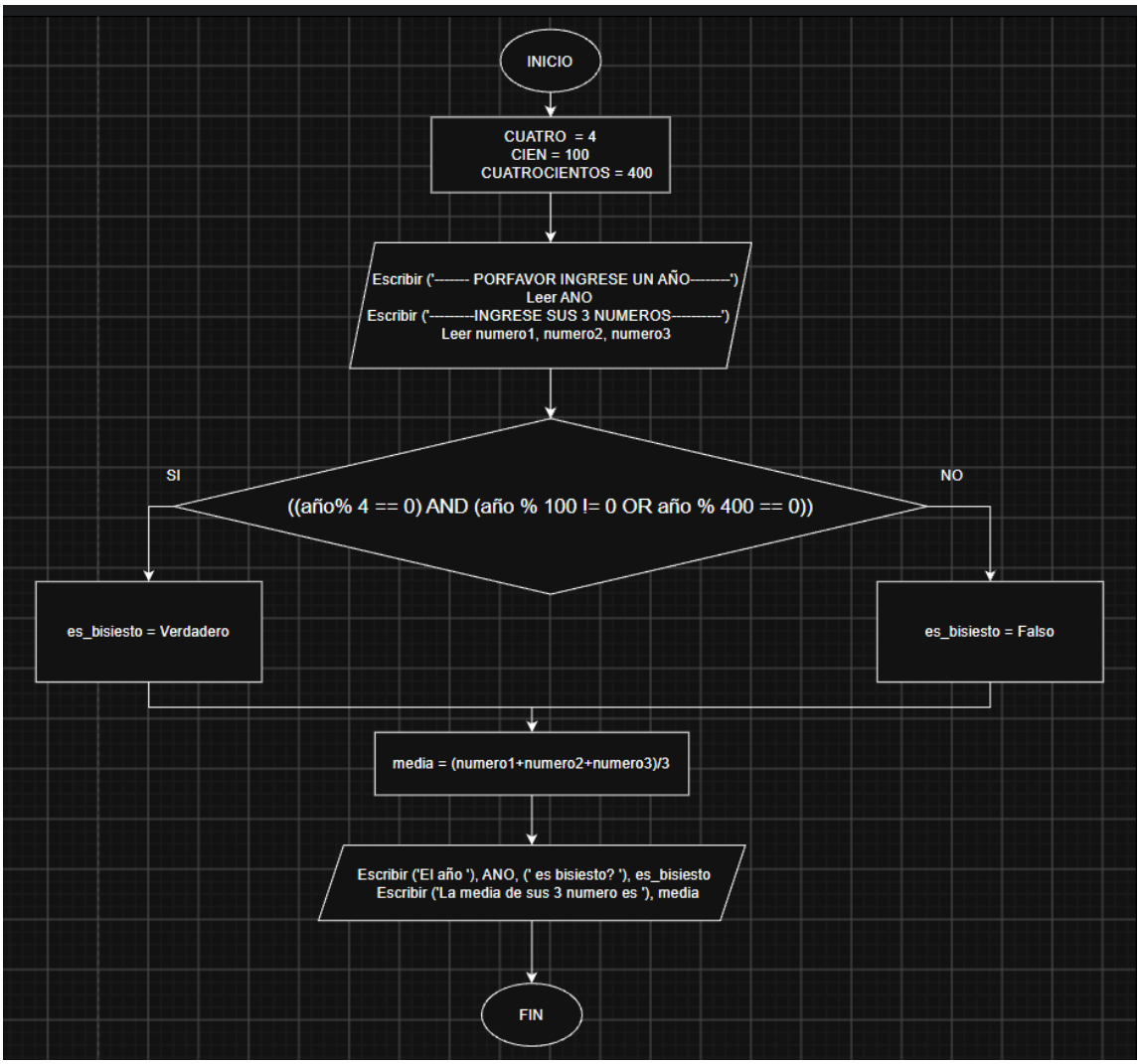


Fig. 57. Diagrama de flujo del décimo quinto ejercicio.

Pseudocódigo

```
Algoritmo cinco
    // Definir variables
    Definir ANO, numero1, numero2, numero3 Como Entero
    Definir CUATRO, CIEN, CUATROCIENTOS Como Entero
    Definir media Como Real
    Definir es_bisiesto Como Lógico
    // Costantes
    CUATRO = 4
    CIEN = 100
    CUATROCIENTOS = 400
    // PEDIR EL AÑO Y LOS 3 NUMERO AL USUARIO
    Escribir ('------- PORFAVOR INGRESE UN AÑO--------')
    Leer ANO
    Escribir ('---------INGRESE SUS 3 NUMEROS---------')
    Leer numero1, numero2, numero3
    // Evaluar la expresion logica
    Si ((año % 4 == 0) y (año % 100 ≠ 0 o año % 400 == 0)) Entonces
        es_bisiesto = Verdadero
    SiNo
        es_bisiesto = Falso
    FinSi
    // La media de 3 numeros
    media = (numero1+numero2+numero3)/3
    // presentar en la pantalla del usuario
    Escribir ('El año '), ANO, (' es bisiesto? '), es_bisiesto
    Escribir ('La media de sus 3 numero es '), media
FinAlgoritmo

```

Fig. 58. Pseudocódigo del décimo quinto ejercicio.

Prueba de escritorio

Paso	cuatro	cien	cuatrocientos	año	numero1	numero2	numero3	es_bisiesto	media	Proceso	Pantalla
1	4	100	400								
2				2024						Lectura de variable año	------- PORFAVOR INGRESE UN AÑO--------
3					10	20	30			Lectura de variables numero1, numero2, numero3	---------INGRESE SUS 3 NUMEROS---------
4								VERDADER		(2024 % 4 == 0 Y 2024 % 100 != 0) O 2024 % 400 == 0	
5									20	media = (10 + 20 + 30) / 3	
6										Impresión de resultados finales	El año 2024 es bisiesto? VERDADERO La media de sus 3 numero es 20

Tabla 29. Primera prueba de escritorio del décimo quinto ejercicio.

Paso	cuatro	cien	cuatrocientos	año	numero1	numero2	numero3	es_bisiesto	media	Proceso	Pantalla
1	4	100	400								
2				2023						Lectura de variable año	------- PORFAVOR INGRESE UN AÑO--------
3					5	10	15			Lectura de variables numero1, numero2, numero3	---------INGRESE SUS 3 NUMEROS---------
4								FALSO		(2023 % 4 == 0 Y 2023 % 100 != 0) O 2023 % 400 == 0	
5									10	media = (5 + 10 + 15) / 3	
6										Impresión de resultados finales	El año 2023 es bisiesto? FALSO La media de sus 3 numero es 10

Tabla 30. Segunda prueba de escritorio del décimo quinto ejercicio.

Codificación en lenguaje C

cincooo.c > main()

```c
#include <stdio.h>
// LIBRERIA PARA DECLARAR BOOLEANOS
#include <stdbool.h>

int main (){
//Definir variables
float numero1, numero2, numero3 ;
int year , CUATRO, CIEN, CUATROCIENTOS ;
float media ;
bool es_bisiesto ;

//constantes
CUATRO = 4 , CIEN = 100 , CUATROCIENTOS = 400;

//PEDIR EL AÑO Y LOS 3 NUMEROS
printf("------- PORFAVOR INGRESE UN year--------\n");
scanf ("%d" , &year );
printf("---------INGRESE SUS 3 NUMEROS----------\n");
scanf("%f %f %f" , &numero1 , &numero2 ,&numero3);

// Evaluar la expresion logica
    if ((year % CUATRO==0) && (year % CIEN!=0 || year % CUATROCIENTOS==0)){
        es_bisiesto = true;
    }else {es_bisiesto = false;}

//La media de 3 numeros
media = (numero1+numero2+numero3)/3;

//presentar en la pantalla al usuario
printf("El year %d es bisiesto? %s\n" , year , es_bisiesto ? "Verdadero" : "Falso");
printf("La media de sus 3 numeros es %.2f \n" , media );

return 0;

}
```

Fig. 59. Programación en C del décimo quinto ejercicio.

Salida del programa

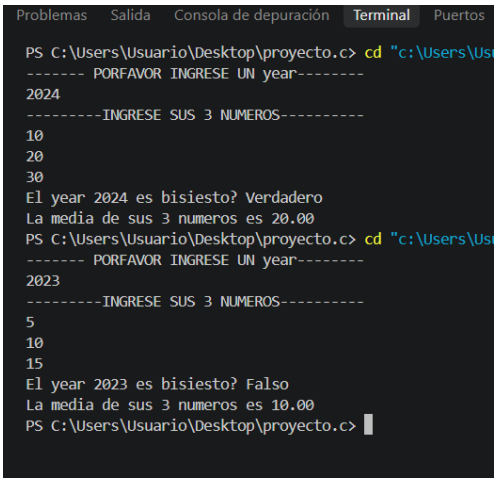


Fig. 60. Salida del programa del décimo quinto ejercicio.

Conclusiones del algoritmo

Con este ejercicio aprendí a programar la lógica para saber si un año es bisiesto o no. Entendí que no basta con ver si el año se puede dividir entre 4, sino que también hay que revisar los años que terminan en 00 para que la respuesta no salga mal.

El AND me sirve para exigir que el año sea divisible para 4 pero también no lo sea para 100. Y el OR se usa para la regla especial si el año sí se puede dividir para 400, automáticamente pasa la prueba y se marca como bisiesto.

Difíciles

EJ- 376

Enunciado del algoritmo

En una cafetería, construya un algoritmo que calcule el impuesto sobre la renta progresivo. Lea el ingreso anual (real). Use constantes simbólicas TRAMO_1 = 10000, TRAMO_2 = 30000, TRAMO_3 = 60000, TASA_1 = 0.10, TASA_2 = 0.15, TASA_3 = 0.20, TASA_4 = 0.30. Evalúe las expresiones lógicas relacionales para determinar los tramos aplicables. Calcule el impuesto con expresiones aritméticas por tramo. Asígnelo a impuesto_total. Muestre por pantalla el desglose. Adicionalmente, calcule la mitad del resultado obtenido. Incluya una sección de conclusiones al final del algoritmo.

Análisis del problema

Datos de entrada

Leer ingreso_anual por teclado tipo real

Proceso

Constantes simbólicas TRAMO_1 = 10000.0, TRAMO_2 = 30000.0, TRAMO_3 = 60000.0 -TASA_1 = 0.10, TASA_2 = 0.15, TASA_3 = 0.20, TASA_4 = 0.30 de tipo real

Evaluar la condición del ingreso para que sea mayor a 0 (ingreso_anual > 0)

Evaluar las condiciones por tramos para calcular impuesto_total

Calcular la mitad del impuesto acumulado

Datos de salida

impuesto_total = el calculo dependiendo de las condiciones que se aplica en el proceso definido como real

mitad_impuesto = Valor de la mitad del impuesto calculado definido como real

Diagrama de flujo

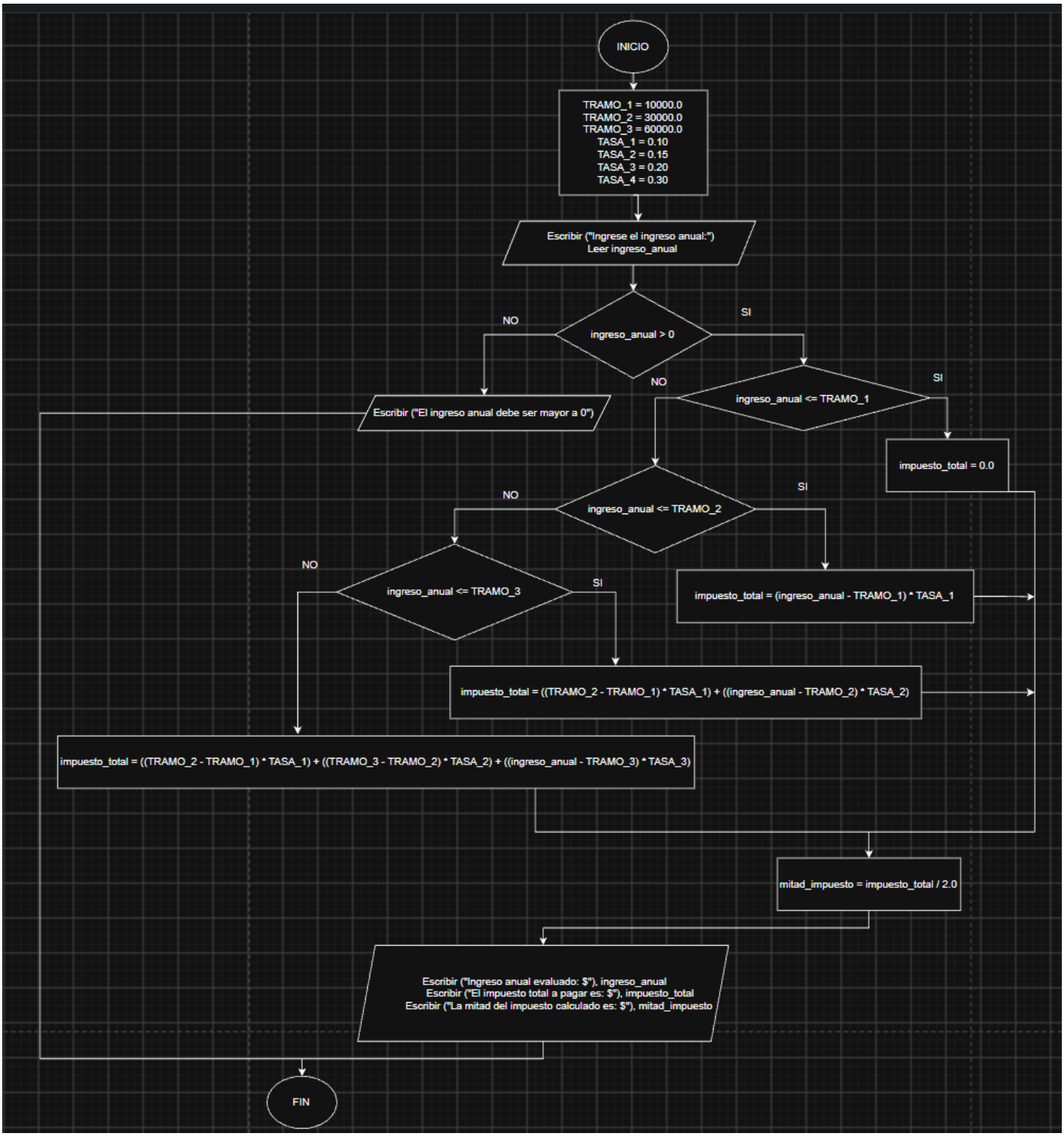


Fig. 61. Diagrama de flujo del décimo sexto ejercicio.

Pseudocódigo

```
Algoritmo dieciseis
    // DEFINICION DE VARIABLES
    Definir ingreso_anual Como Real
    Definir impuesto_total Como Real
    Definir mitad_impuesto Como Real
    Definir TRAMO_1, TRAMO_2, TRAMO_3 Como Real
    Definir TASA_1, TASA_2, TASA_3, TASA_4 Como Real

    // CONSTANTES
    TRAMO_1 = 10000.0
    TRAMO_2 = 30000.0
    TRAMO_3 = 60000.0
    TASA_1 = 0.10
    TASA_2 = 0.15
    TASA_3 = 0.20
    TASA_4 = 0.30
    // LECTURA POR TECLADO
    Escribir ("Ingrese el ingreso anual:")
    Leer ingreso_anual

    // PROCESO Y VALIDACION
    Si (ingreso_anual > 0) Entonces
        Si (ingreso_anual ≤ TRAMO_1) Entonces
            impuesto_total = 0.0
        Sino
            Si (ingreso_anual ≤ TRAMO_2) Entonces
                impuesto_total = (ingreso_anual - TRAMO_1) * TASA_1
            Sino
                Si (ingreso_anual ≤ TRAMO_3) Entonces
                    impuesto_total = ((TRAMO_2 - TRAMO_1) * TASA_1) + ((ingreso_anual - TRAMO_2) * TASA_2)
                Sino
                    impuesto_total = ((TRAMO_2 - TRAMO_1) * TASA_1) + ((TRAMO_3 - TRAMO_2) * TASA_2) + ((ingreso_anual - TRAMO_3) * TASA_3)
                FinSi
            FinSi
        FinSi

        mitad_impuesto = impuesto_total / 2.0

        // PRESENTACION AL USUARIO
        Escribir ("Ingreso anual evaluado: $"), ingreso_anual
        Escribir ("El impuesto total a pagar es: $"), impuesto_total
        Escribir ("La mitad del impuesto calculado es: $"), mitad_impuesto
    Sino
        Escribir ("El ingreso anual debe ser mayor a 0")
    FinSi
FinAlgoritmo
```

Fig. 62. Pseudocódigo del décimo sexto ejercicio.

Prueba de escritorio

Paso	TRAMO_1	TRAMO_2	TRAMO_3	TASA_1	TASA_2	TASA_3	TASA_4	ingreso_anual	impuesto_total	mitad_impuesto	Proceso	Pantalla
1	10000	30000	60000	0.1	0.15	0.2	0.3					
2								25000			Lectura de variable ingreso_anual	Ingrese el ingreso anual:
3									1500		impuesto_total = (25000.0 - 10000.0) * 0.10	
4										750	mitad_impuesto = 1500.0 / 2.0	
5											ingreso_anual > 0 -> VERDADERO	Ingreso anual evaluado: $25000 El impuesto total a pagar es: $1500 La mitad del impuesto calculado es: $750

Tabla 31. Primera prueba de escritorio del décimo sexto ejercicio.

Paso	TRAMO_1	TRAMO_2	TRAMO_3	TASA_1	TASA_2	TASA_3	TASA_4	ingreso_anual	impuesto_total	mitad_impuesto	Proceso	Pantalla
1	10000	30000	60000	0.1	0.15	0.2	0.3					
2								45000			Lectura de variable ingreso_anual	Ingrese el ingreso anual:
3									4250		impuesto_total = (20000.0 * 0.10) + (15000.0 * 0.15)	
4										2125	mitad_impuesto = 4250.0 / 2.0	
5											ingreso_anual > 0 -> VERDADERO	Ingreso anual evaluado: $45000 El impuesto total a pagar es: $4250 La mitad del impuesto calculado es: $2125

Tabla 32. Segunda prueba de escritorio del décimo sexto ejercicio.

Codificación en lenguaje C

```c
C DIFICILuno.c > main()
#include <stdio.h>

int main (){
// DEFINICION DE VARIABLES
float ingreso_anual , impuesto_total , mitad_impuesto;

// CONSTANTES
const float TRAMO_1 = 10000.0 , TRAMO_2 = 30000.0 , TRAMO_3 = 60000.0 , TASA_1 = 0.10 , TASA_2 = 0.15 , TASA_3 = 0.20 , TASA_4 = 0.30;

// LEER POR TECLADO
printf ("Ingrese el ingreso anual \n");
    scanf ("%f" , &ingreso_anual);

// proceso de resolucion
if (ingreso_anual > 0){
    if (ingreso_anual <= TRAMO_1){
        impuesto_total = 0.0;
    }else
         if (ingreso_anual <= TRAMO_2){
        impuesto_total = (ingreso_anual - TRAMO_1) * TASA_1;
         }else
             if (ingreso_anual <= TRAMO_3){
               impuesto_total = ((TRAMO_2 - TRAMO_1) * TASA_1) + ((ingreso_anual - TRAMO_2) * TASA_2);
             }else{
                impuesto_total = ((TRAMO_2 - TRAMO_1) * TASA_1) + ((TRAMO_3 - TRAMO_2) * TASA_2) + ((ingreso_anual - TRAMO_3) * TASA_3);
             }


    mitad_impuesto = impuesto_total / 2.0;
    // PRESENTAR PANTALLA
    printf ("Ingreso anual evaluado: %.2f \n" , ingreso_anual);
    printf ("El impuesto total a pagar es: %.2f \n" , impuesto_total);
    printf ("La mitad del impuesto calculado es: %.2f \n" , mitad_impuesto);

}else{ printf("El ingreso anual debe ser mayor a 0 \n");}

return 0;
}
```

Fig. 63. Programación en C del décimo sexto ejercicio.

Salida del programa

```
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuario
Ingrese el ingreso anual
25000
Ingreso anual evaluado: 25000.00
El impuesto total a pagar es: 1500.00
La mitad del impuesto calculado es: 750.00
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Users\Usuario
Ingrese el ingreso anual
45000
Ingreso anual evaluado: 45000.00
El impuesto total a pagar es: 4250.00
La mitad del impuesto calculado es: 2125.00
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 64. Salida del programa del décimo sexto ejercicio.

Conclusiones del algoritmo

Comprendí cómo utilizar la estructura de los condicionales (if-else) para aplicar diferentes a las diferentes tasas porcentuales únicamente sobre los montos excedentes de cada tramo.

Aprendí que la palabra reservada const y el tipo de dato deben colocarse únicamente en la declaración inicial de la constante, ya que si se declara dos veces dará error al copilar.

EJ- 377

Enunciado del algoritmo

En una florería, cree un algoritmo que simule un sistema de puntuación. Lea los puntos obtenidos (entero), el tiempo jugado (real, en minutos) y el nivel alcanzado (entero). Use constantes simbólicas BONO_TIEMPO = 0.10, BONO_NIVEL = 100, TIEMPO_MIN = 30.0, NIVEL_MAX = 10. Calcule: bono_time = puntos * BONO_TIEMPO * (tiempo < TIEMPO_MIN ? 1 : 0); bono_nivel = nivel * BONO_NIVEL; puntuacion_total = puntos + bono_time + bono_nivel. Evalúe la expresión lógica (nivel >= 1 AND nivel <= NIVEL_MAX). Muestre por pantalla la puntuación total. Adicionalmente, calcule la mitad del resultado obtenido. Verifique que el algoritmo funcione correctamente con valores limite.

Análisis del problema

Datos de entrada

Leer puntos por teclado tipo entero

Leer tiempo por teclado tipo real en minutos

Leer nivel por teclado tipo entero

Proceso

BONO_TIEMPO = 0.10 Tipo Real

BONO_NIVEL = 100 Tipo Entero

TIEMPO_MIN = 30.0 Tipo Real

NIVEL_MAX = 10 Tipo Entero

Evaluar la condición de validación de nivel (nivel >= 1 Y nivel <= NIVEL_MAX) asignada a nivel_valido

Calcular bono por tiempo bono_time = puntos * BONO_TIEMPO * (tiempo < TIEMPO_MIN ? 1 : 0)

Calcular bono por nivel bono_nivel = nivel * BONO_NIVEL

Calcular puntuación total puntuacion_total = puntos + bono_time + bono_nivel

Calcular la mitad de la puntuación total mitad_puntuacion = puntuacion_total / 2.0

Datos de salida

puntuacion_total = Resultado final de la suma de puntos y bonos definido como real

mitad_puntuacion = Resultado de dividir la puntuación total entre dos definidos como real

Diagrama de flujo

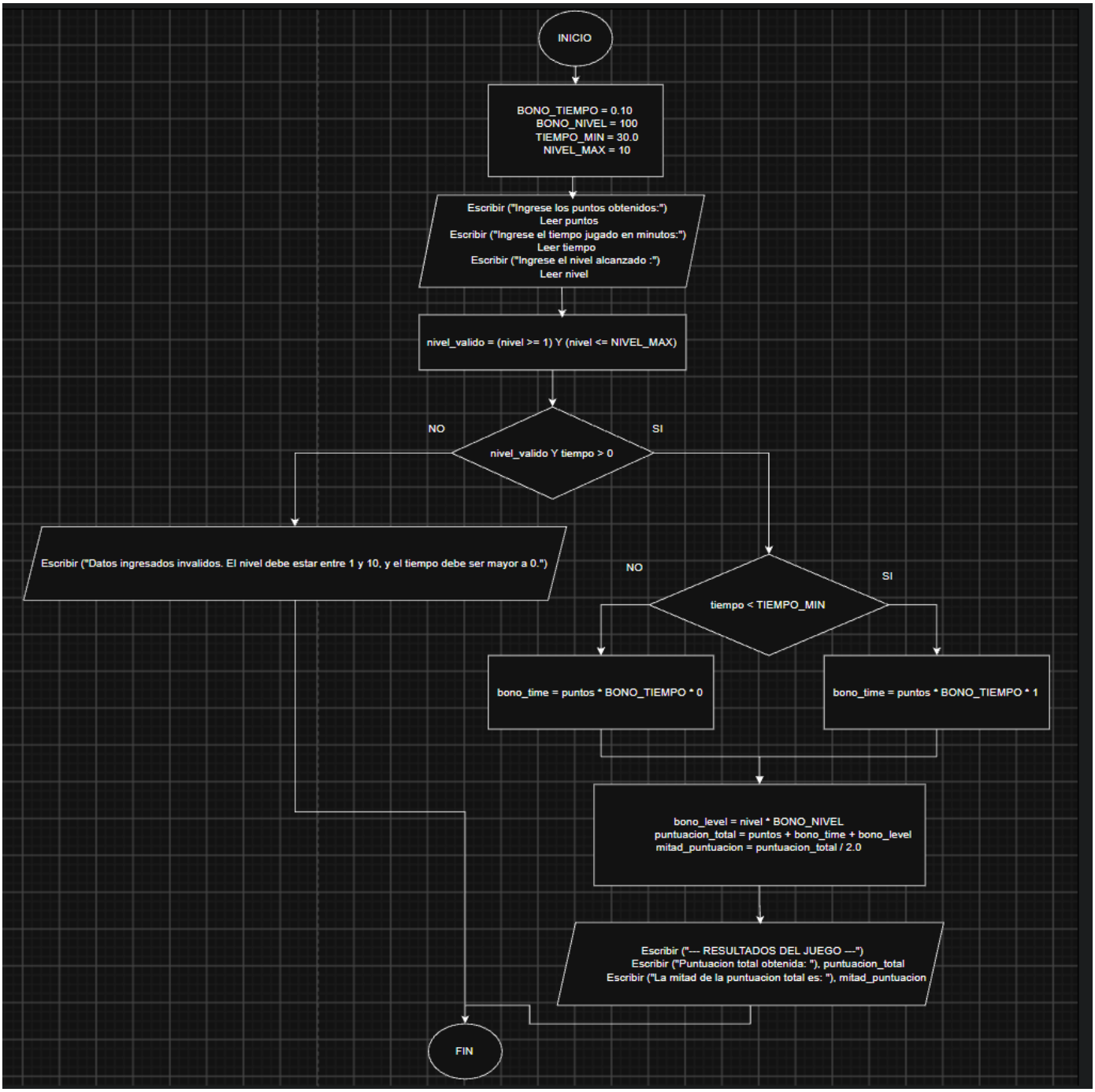


Fig. 65. Diagrama de flujo del décimo séptimo ejercicio.

Pseudocódigo

```
Algoritmo diecisiete
    // DEFINICION DE VARIABLES
    Definir puntos, nivel Como Entero
    Definir tiempo, bono_time, bono_level, puntuacion_total, mitad_puntuacion Como Real
    Definir nivel_valido Como Logico
    Definir BONO_TIEMPO, TIEMPO_MIN Como Real
    Definir BONO_NIVEL, NIVEL_MAX Como Entero
    // CONSTANTES
    BONO_TIEMPO = 0.10
    BONO_NIVEL = 100
    TIEMPO_MIN = 30.0
    NIVEL_MAX = 10

    // LECTURA POR TECLADO
    Escribir ("Ingrese los puntos obtenidos:")
    Leer puntos
    Escribir ("Ingrese el tiempo jugado en minutos:")
    Leer tiempo
    Escribir ("Ingrese el nivel alcanzado :")
    Leer nivel
    // VALIDACION DE NIVEL Y TIEMPO
    nivel_valido = (nivel ≥ 1) Y (nivel ≤ NIVEL_MAX)

    Si nivel_valido Y tiempo > 0 Entonces

        // EVALUACION DEL BONO POR TIEMPO
        Si tiempo < TIEMPO_MIN Entonces
            bono_time = puntos * BONO_TIEMPO * 1
        Sino
            bono_time = puntos * BONO_TIEMPO * 0
        FinSi
        // CALCULOS DE PUNTUACION
        bono_level = nivel * BONO_NIVEL
        puntuacion_total = puntos + bono_time + bono_level
        mitad_puntuacion = puntuacion_total / 2.0

        // PRESENTACION AL USUARIO
        Escribir ("--- RESULTADOS DEL JUEGO ---")
        Escribir ("Puntuacion total obtenida: "), puntuacion_total
        Escribir ("La mitad de la puntuacion total es: "), mitad_puntuacion
    Sino
        Escribir ("Datos ingresados invalidos. El nivel debe estar entre 1 y 10, y el tiempo debe ser mayor a 0.")
    FinSi
FinAlgoritmo
```

Fig. 66. Pseudocódigo del décimo séptimo ejercicio.

Prueba de escritorio

Paso	BONO_TIEMPO	BONO_NIVEL	TIEMPO_MIN	NIVEL_MAX	puntos	tiempo	nivel	nivel_valido	bono_time	bono_level	puntuacion_total	mitad_puntuacion	Proceso	Pantalla
1	0.1	100	30	10										
2					500								Lectura de variable puntos	Ingrese los puntos obtenidos:
3						20							Lectura de variable tiempo	Ingrese el tiempo jugado en minutos:
4							5						Lectura de variable nivel	Ingrese el nivel alcanzado :
5								VERDADERO					(5 >= 1) Y (5 <= 10)	
6									50				tiempo < 30.0 -> 500 * 0.10 * 1	
7										500			bono_level = 5 * 100	
8											1050		puntuacion_total = 500 + 50.0 + 500.0	
9												525	mitad_puntuacion = 1050.0 / 2.0	
10													nivel_valido Y tiempo > 0 -> VERDADERO	Puntuacion total obtenida: 1050 La mitad de la puntuacion total es: 525

Tabla 33. Primera prueba de escritorio del décimo séptimo ejercicio.

Paso	BONO_TIEMPO	BONO_NIVEL	TIEMPO_MIN	NIVEL_MAX	puntos	tiempo	nivel	nivel_valido	bono_time	bono_level	puntuacion_total	mitad_puntuacion	Proceso	Pantalla
1	0.1	100	30	10										
2					400								Lectura de variable puntos	Ingrese los puntos obtenidos:
3						45							Lectura de variable tiempo	Ingrese el tiempo jugado en minutos:
4							3						Lectura de variable nivel	Ingrese el nivel alcanzado :
5								VERDADERO					(3 >= 1) Y (3 <= 10)	
6									0				tiempo >= 30.0 -> 400 * 0.10 * 0	
7										300			bono_level = 3 * 100	
8											700		puntuacion_total = 400 + 0.0 + 300.0	
9												350	mitad_puntuacion = 700.0 / 2.0	
10													nivel_valido Y tiempo > 0 -> VERDADERO	Puntuacion total obtenida: 700 La mitad de la puntuacion total es: 350

Tabla 34. Segunda prueba de escritorio del décimo séptimo ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>
#include <stdbool.h>

int main(){
// DEFINICION DE VARIABLES
int puntos, nivel , BONO_NIVEL, NIVEL_MAX ;
float tiempo, bono_time, bono_level, puntuacion_total, mitad_puntuacion ;
bool nivel_valido;
float BONO_TIEMPO, TIEMPO_MIN;

// CONSTANTES
BONO_TIEMPO = 0.10 , BONO_NIVEL = 100 ,TIEMPO_MIN = 30.0 , NIVEL_MAX = 10;

// LEER POR TECLADO
printf ("Ingrese los puntos obtenidos \n");
    scanf ("%d" , &puntos);
printf ("Ingrese el tiempo en minutos \n");
    scanf ("%f" , &tiempo);
printf ("Ingrese el nivel alcanzado \n");
    scanf ("%d" , &nivel);

//validacion de niver y tiempo
nivel_valido = nivel >= 1 && nivel <= NIVEL_MAX;

if (nivel_valido && tiempo > 0){

    //Evaluacion de bono por tiempo
    if (nivel_valido && tiempo > 0){
        bono_time = puntos * BONO_TIEMPO * 1;
    }else{
        bono_time = puntos * BONO_TIEMPO * 0;
    }

    //calculo de puntuacion
    bono_level = nivel * BONO_NIVEL;
    puntuacion_total = puntos + bono_time + bono_level;
    mitad_puntuacion = puntuacion_total / 2.0;

    // PRESENTAR PANTALLA
    printf ("-- RESULTADOS DEL JUEGO --\n");
    printf ("Puntuacion total obtenida: %.2f \n" , puntuacion_total);
    printf ("La mitad de la puntuacion total es: %.2f \n" , mitad_puntuacion);

}else{ printf("El nivel debe estar entre 1 y 10 y el tiempo debe ser mayor a 0 \n");}

return 0;

}
```

Fig. 67. Programación en C del décimo séptimo ejercicio.

Salida del programa

```
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Use
Ingrese los puntos obtenidos
500
Ingrese el tiempo en minutos
20
Ingrese el nivel alcanzado
5
-- RESULTADOS DEL JUEGO --
Puntuacion total obtenida: 1050.00
La mitad de la puntuacion total es: 525.00
PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:\Use
Ingrese los puntos obtenidos
400
Ingrese el tiempo en minutos
45
Ingrese el nivel alcanzado
3
-- RESULTADOS DEL JUEGO --
Puntuacion total obtenida: 740.00
La mitad de la puntuacion total es: 370.00
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 68. Salida del programa del décimo séptimo ejercicio.

Conclusiones del algoritmo

Comprendí cómo combinar números enteros y decimales en las operaciones para que los puntos extra con porcentajes se calculen de forma exacta.

Con el algoritmo aprendí a utilizar de mejor manera las condiciones para que mi algoritmo me de unas respuestas satisfactorias.

EJ- 378

Enunciado del algoritmo

En una aseguradora, diseñe un algoritmo que valide el numero de una tarjeta de crédito (cadena de 16 dígitos) usando el algoritmo de Luhn. Use constantes simbólicas LONGITUD_TARJETA = 16, MODULO = 10. Use una variable entera suma inicializada con literal 0. Aplique el algoritmo: duplicar los dígitos en posiciones pares, sumar los dígitos de los resultados. Calcule la suma total con expresiones aritméticas. Evalúe la expresión lógica (suma % MODULO == 0). Asigne el resultado a tarjeta_valida. Muestre por pantalla la validación. Adicionalmente, calcule la mitad del resultado obtenido. Agregue al inicio una descripción breve de los pasos a seguir.

Análisis del problema

Datos de entrada

Leer numero_tarjeta por teclado tipo Cadena texto de 16 dígitos

Proceso

Constante LONGITUD_TARJETA = 16 Tipo Entero

Constante MODULO = 10 Tipo Entero

Iniciar la variable suma = 0

Verificar que el texto ingresado tenga 16 números.

Si el resultado es mayor a 9, restarle 9 para que sea de un solo dígito.

Multiplicar por 2 los números en posiciones pares

Ir sumando cada resultado dentro de la variable suma

Evalúe la expresión lógica (suma % MODULO == 0) y signe el resultado a tarjeta_valida

Calcular la mitad del total mitad_suma = suma / 2.0.

Datos de salida

tarjeta_valida : el mensaje indicando si la tarjeta es válida o no.

suma: El valor del conteo total.

mitad_suma: La mitad del valor total obtenido.

Diagrama de flujo

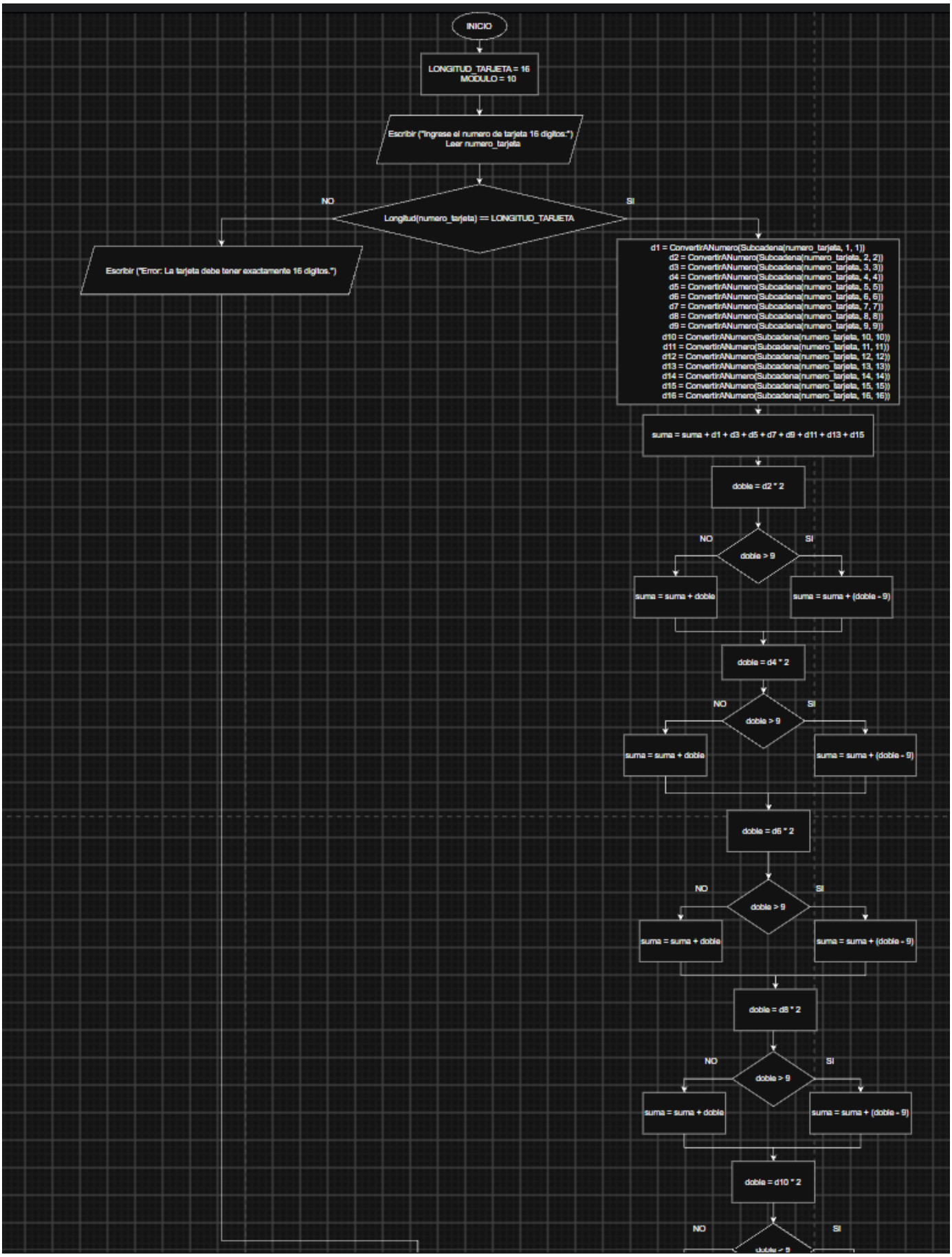


Fig. 69. Diagrama de flujo Primera parte del décimo octavo ejercicio.

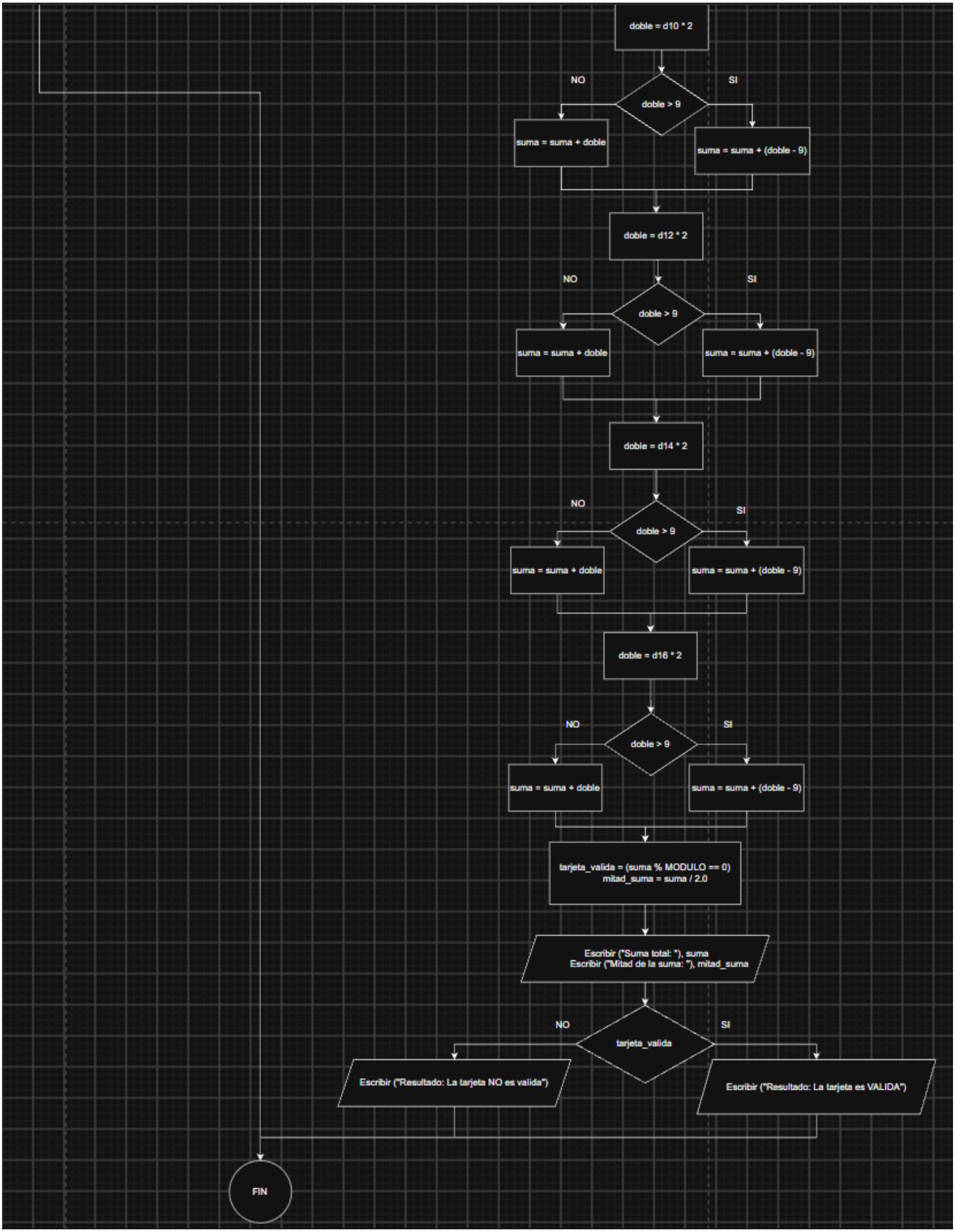


Fig. 70. Diagrama de flujo Segunda parte del décimo octavo ejercicio.

Pseudocódigo

```
Algoritmo dieciocho
    // DEFINICION DE VARIABLES
    Definir numero_tarjeta Como Cadena
    Definir d1, d2, d3, d4, d5, d6, d7, d8, d9, d10, d11, d12, d13, d14, d15, d16 Como Entero
    Definir doble, suma Como Entero
    Definir tarjeta_valida Como Logico
    Definir mitad_suma Como Real
    Definir LONGITUD_TARJETA, MODULO Como Entero

    // CONSTANTES
    LONGITUD_TARJETA = 16
    MODULO = 10

    // INICIALIZACION
    suma = 0

    // ENTRADA DE DATOS
    Escribir ("Ingrese el numero de tarjeta 16 digitos:")
    Leer numero_tarjeta

    // PROCESO Y VALIDACION
    Si Longitud(numero_tarjeta) == LONGITUD_TARJETA Entonces

        // EXTRAER CADA DIGITO
        d1 = ConvertirANumero(Subcadena(numero_tarjeta, 1, 1))
        d2 = ConvertirANumero(Subcadena(numero_tarjeta, 2, 2))
        d3 = ConvertirANumero(Subcadena(numero_tarjeta, 3, 3))
        d4 = ConvertirANumero(Subcadena(numero_tarjeta, 4, 4))
        d5 = ConvertirANumero(Subcadena(numero_tarjeta, 5, 5))
        d6 = ConvertirANumero(Subcadena(numero_tarjeta, 6, 6))
        d7 = ConvertirANumero(Subcadena(numero_tarjeta, 7, 7))
        d8 = ConvertirANumero(Subcadena(numero_tarjeta, 8, 8))
        d9 = ConvertirANumero(Subcadena(numero_tarjeta, 9, 9))
        d10 = ConvertirANumero(Subcadena(numero_tarjeta, 10, 10))
        d11 = ConvertirANumero(Subcadena(numero_tarjeta, 11, 11))
        d12 = ConvertirANumero(Subcadena(numero_tarjeta, 12, 12))
        d13 = ConvertirANumero(Subcadena(numero_tarjeta, 13, 13))
        d14 = ConvertirANumero(Subcadena(numero_tarjeta, 14, 14))
        d15 = ConvertirANumero(Subcadena(numero_tarjeta, 15, 15))
        d16 = ConvertirANumero(Subcadena(numero_tarjeta, 16, 16))

        // SUMAR POSICIONES IMPARES
        suma = suma + d1 + d3 + d5 + d7 + d9 + d11 + d13 + d15

        // Posicion 2
        doble = d2 * 2
        Si doble > 9 Entonces
            suma = suma + (doble - 9)
        Sino
            suma = suma + doble
        FinSi

        // Posicion 4
        doble = d4 * 2
        Si doble > 9 Entonces
            suma = suma + (doble - 9)
        Sino
            suma = suma + doble
        FinSi

        // Posicion 6
        doble = d6 * 2
```

Fig. 71. Pseudocódigo Primera parte del décimo octavo ejercicio.

```
62          doble = d6 * 2
63          Si doble > 9 Entonces
64              suma = suma + (doble - 9)
65          Sino
66              suma = suma + doble
67          FinSi
68
69          // Posicion 8
70          doble = d8 * 2
71          Si doble > 9 Entonces
72              suma = suma + (doble - 9)
73          Sino
74              suma = suma + doble
75          FinSi
76
77          // Posicion 10
78          doble = d10 * 2
79          Si doble > 9 Entonces
80              suma = suma + (doble - 9)
81          Sino
82              suma = suma + doble
83          FinSi
84
85          // Posicion 12
86          doble = d12 * 2
87          Si doble > 9 Entonces
88              suma = suma + (doble - 9)
89          Sino
90              suma = suma + doble
91          FinSi
92
93          // Posicion 14
94          doble = d14 * 2
95          Si doble > 9 Entonces
96              suma = suma + (doble - 9)
97          Sino
98              suma = suma + doble
99          FinSi
100
101         // Posicion 16
102         doble = d16 * 2
103         Si doble > 9 Entonces
104             suma = suma + (doble - 9)
105         Sino
106             suma = suma + doble
107         FinSi
108         // EVALUAR EXPRESION LOGICA
109         tarjeta_valida = (suma % MODULO == 0)
110         mitad_suma = suma / 2.0
111
112         // SALIDA DE DATOS
113         Escribir ("--- RESULTADOS ---")
114         Escribir ("Suma total: "), suma
115         Escribir ("Mitad de la suma: "), mitad_suma
116
117         Si tarjeta_valida Entonces
118             Escribir ("Resultado: La tarjeta es VALIDA")
119         Sino
120             Escribir ("Resultado: La tarjeta NO es valida")
121         FinSi
122
123     Sino
124         Escribir ("Error: La tarjeta debe tener exactamente 16 digitos.")
125     FinSi
126 FinAlgoritmo
```

Fig. 72. Pseudocódigo (Segunda parte) del décimo octavo ejercicio.

Prueba de escritorio

Paso	LONGITUD_TARJETA	MODULO	numero_tarjeta	d1..d16	suma	doble	tarjeta_valida	mitad_suma	Proceso	Pantalla
1	16	10			0				Inicialización de suma	
2			4E+15						Lectura de numero_tarjeta	Ingrese el numero de tarjeta 16 digitos:
3				d1=4, d16=3					Extracción de dígitos d1 a d16	
4					4				Suma de impares: 4 + 0 + ... + 0	
5					10	6			Suma de pares con doble (d16=3*2=6)	
6							VERDADER	5	10 % 10 == 0 -> VERDADERO | mitad_suma = 10 / 2.0	
7									Impresión de resultados	Suma total: 10 Mitad de la suma: 5 Resultado: La tarjeta es VALIDA

Tabla 35. Primera prueba de escritorio del décimo octavo ejercicio.

Paso	LONGITUD_TARJETA	MODULO	numero_tarjeta	d1..d16	suma	doble	tarjeta_valida	mitad_suma	Proceso	Pantalla
1	16	10			0				Inicialización de suma	
2			123456						Lectura de numero_tarjeta	Ingrese el numero de tarjeta 16 digitos:
3									Longitud(123456) == 16 -> FALSO	Error: La tarjeta debe tener exactamente 16 digitos.

Tabla 36. Segunda prueba de escritorio del décimo octavo ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

int main() {
    // CONSTANTES
    const int LONGITUD_TARJETA = 16;
    const int MODULO = 10;

    // DEFINICION DE VARIABLES
    char numero_tarjeta[20];
    int d1, d2, d3, d4, d5, d6, d7, d8, d9, d10, d11, d12, d13, d14, d15, d16;
    int doble, suma = 0;
    bool tarjeta_valida;
    float mitad_suma;

    // ENTRADA DE DATOS
    printf("Ingrese el numero de tarjeta (16 digitos): ");
    scanf("%s", numero_tarjeta);

    // PROCESO Y VALIDACION
    if (strlen(numero_tarjeta) == LONGITUD_TARJETA) {

        // EXTRAER CADA DIGITO SE LE RESTA 0 PARA CONVERTIRLO A ENTERO
        d1 = numero_tarjeta[0] - '0';
        d2 = numero_tarjeta[1] - '0';
        d3 = numero_tarjeta[2] - '0';
        d4 = numero_tarjeta[3] - '0';
        d5 = numero_tarjeta[4] - '0';
        d6 = numero_tarjeta[5] - '0';
        d7 = numero_tarjeta[6] - '0';
        d8 = numero_tarjeta[7] - '0';
        d9 = numero_tarjeta[8] - '0';
        d10 = numero_tarjeta[9] - '0';
        d11 = numero_tarjeta[10] - '0';
        d12 = numero_tarjeta[11] - '0';
        d13 = numero_tarjeta[12] - '0';
        d14 = numero_tarjeta[13] - '0';
        d15 = numero_tarjeta[14] - '0';
        d16 = numero_tarjeta[15] - '0';

        // SUMAR POSICIONES IMPARES
        suma = suma + d1 + d3 + d5 + d7 + d9 + d11 + d13 + d15;

        // EVALUAR CADA POSICION
        // Posicion 2
        doble = d2 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 4
        doble = d4 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 6
        doble = d6 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 8
        doble = d8 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 10
        doble = d10 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 12
        doble = d12 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 14
        doble = d14 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // Posicion 16
        doble = d16 * 2;
        if (doble > 9) {
            suma = suma + (doble - 9);
        } else {
            suma = suma + doble;
        }

        // EVALUAR EXPRESION LOGICA
        tarjeta_valida = (suma % MODULO == 0);
        mitad_suma = suma / 2.0;

        // SALIDA DE DATOS
        printf("\n--- RESULTADOS ---\n");
        printf("Suma total: %d\n", suma);
        printf("Mitad de la suma: %.2f\n", mitad_suma);

        if (tarjeta_valida) {
            printf("Resultado: La tarjeta es VALIDA\n");
        } else {
            printf("Resultado: La tarjeta NO es valida\n");
        }

    } else {
        printf("Error: La tarjeta debe tener exactamente 16 digitos.\n");
    }

    return 0;
}
```

Fig. 73. Programación en C del décimo octavo ejercicio.

Salida del programa

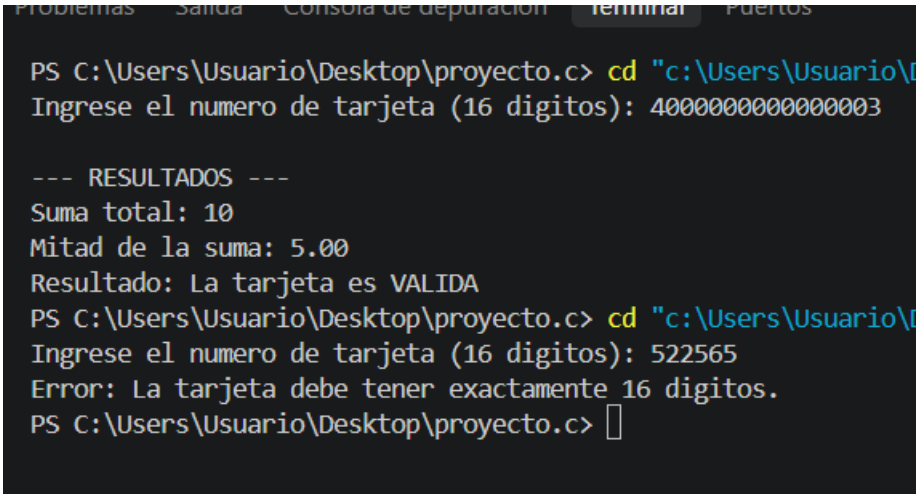


Fig. 74. Salida del programa del décimo octavo ejercicio.

Conclusiones del algoritmo

La librería <string.h> la use para la función strlen(), que permite contar los caracteres ingresados y verificar que la tarjeta tenga 16 dígitos antes de procesarla.

Al no poder usar bucles, tuve que extraer cada dígito individual usando -'0' para convertirlo a número y aplicar un if-else a cada posición par para duplicarla y restarle 9 si pasaba de 9, así pude hacer que mi algoritmo se ejecute bien.

EJ- 379

Enunciado del algoritmo

En un hotel, construya un algoritmo que lea el radio y la altura de un cilindro (dos reales). Use constantes simbólicas PI = 3.14159. Calcule: area_base = PI * radio * radio; area_lateral = 2 * PI * radio * altura; area_total = 2 * area_base + area_lateral; volumen = area_base * altura. Asigne los resultados a variables reales. Evalúe la expresión lógica (radio > 0 AND altura > 0). Muestre por pantalla todas las medidas. Adicionalmente, calcule la mitad del resultado obtenido. Documente cada paso del algoritmo con comentarios explicativos.

Análisis del problema

Datos de entrada

Leer por teclado el radio del cilindro tipo Real.

Leer por teclado la altura del cilindro tipo Real.

Proceso

Constante PI = 3.14159 Tipo Real.

Evaluar la expresión lógica (radio > 0 AND altura > 0) para poder ejecutar el resto del programa

Calcular area_base = PI * radio * radio.

Calcular area_lateral = 2 * PI * radio * altura.

Calcular area_total = 2 * area_base + area_lateral.

Calcular volumen = area_base * altura.

Calcular las mitades de las respuestas

mitad_area_base = area_base / 2.0

mitad_area_lateral = area_lateral / 2.0

mitad_area_total = area_total / 2.0

mitad_volumen = volumen / 2.0

Datos de salida

area_base y mitad_area_base según los cálculos del programa

area_lateral y mitad_area_lateral según los cálculos del programa

area_total y mitad_area_total según los cálculos del programa

volumen y mitad_volumen según los cálculos del programa

Diagrama de flujo

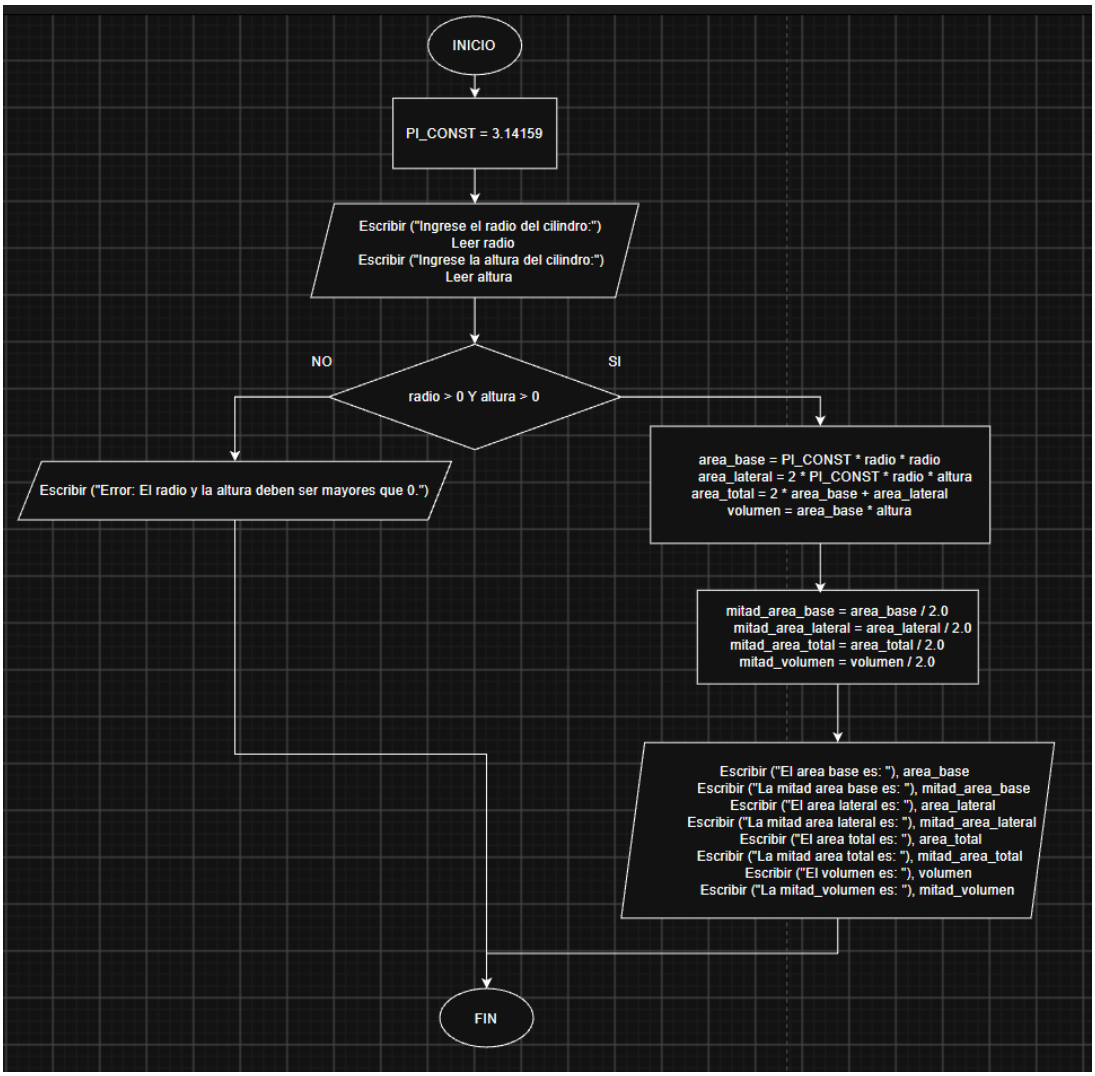


Fig. 75. Diagrama de flujo del décimo noveno ejercicio.

Pseudocódigo

```
Algoritmo dicinueve
    // DEFINICION DE VARIABLES
    Definir radio, altura Como Real
    Definir area_base, area_lateral, area_total, volumen Como Real
    Definir mitad_area_base, mitad_area_lateral, mitad_area_total, mitad_volumen Como Real
    Definir PI_CONST Como Real

    // CONSTANTE
    PI_CONST = 3.14159

    // LEER POR TECLADO
    Escribir ("Ingrese el radio del cilindro:")
    Leer radio

    Escribir ("Ingrese la altura del cilindro:")
    Leer altura

    // EVALUAR EXPRESION LOGICA
    Si (radio > 0 Y altura > 0) Entonces

        // CALCULOS
        area_base = PI_CONST * radio * radio
        area_lateral = 2 * PI_CONST * radio * altura
        area_total = 2 * area_base + area_lateral
        volumen = area_base * altura

        // CALCULAR LAS MITADES DE LAS RESPUESTAS
        mitad_area_base = area_base / 2.0
        mitad_area_lateral = area_lateral / 2.0
        mitad_area_total = area_total / 2.0
        mitad_volumen = volumen / 2.0

        // PRESENTAR AL USUARIO
        Escribir ("--- RESULTADOS DEL CILINDRO ---")
        Escribir ("El area base es: "), area_base
        Escribir ("La mitad area base es: "), mitad_area_base

        Escribir ("El area lateral es: "), area_lateral
        Escribir ("La mitad area lateral es: "), mitad_area_lateral

        Escribir ("El area total es: "), area_total
        Escribir ("La mitad area total es: "), mitad_area_total

        Escribir ("El volumen es: "), volumen
        Escribir ("La mitad_volumen es: "), mitad_volumen

    Sino
        Escribir ("Error: El radio y la altura deben ser mayores que 0.")
    FinSi
FinAlgoritmo
```

Fig. 76. Pseudocódigo del décimo noveno ejercicio.

Prueba de escritorio

Paso	PI_CONST	radio	altura	area_base	mitad_area_base	area_lateral	mitad_area_lateral	area_total	mitad_area_total	volumen	mitad_volumen	Proceso	Pantalla
1	3.14159												
2		3										Lectura de variable radio	Ingrese el radio del cilindro:
3			5									Lectura de variable altura	Ingrese la altura del cilindro:
4				28.27	14.14							area_base = 3.14159 * 3.0 * 3.0	
5						94.25	47.12					area_lateral = 2 * 3.14159 * 3.0 * 5.0	
6								150.8	75.4			area_total = 2 * 28.27 + 94.25	
7										141.37	70.69	volumen = 28.27 * 5.0	
8												radio > 0 Y altura > 0 -> VERDADERO	El area base es: 28.27 La mitad area base es: 14.14 El area lateral es: 94.25 La mitad area lateral es: 47.12 El area total es: 150.80 La mitad area total es: 75.40 El volumen es: 141.37 La mitad_volumen es: 70.69

Tabla 37. Primera prueba de escritorio del décimo noveno ejercicio.

Paso	PI_CONST	radio	altura	area_base	mitad_area_base	area_lateral	mitad_area_lateral	area_total	mitad_area_total	volumen	mitad_volumen	Proceso	Pantalla
1	3.14159												
2		-2										Lectura de variable radio	Ingrese el radio del cilindro:
3			5									Lectura de variable altura	Ingrese la altura del cilindro:
4												radio > 0 Y altura > 0 -> FALSO	Error: El radio y la altura deben ser mayores que 0.

Tabla 38. Segunda prueba de escritorio del décimo noveno ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>

int main() {
    // DEFINICION DE VARIABLES
    float radio, altura;
    float area_base, area_lateral, area_total, volumen;
    float mitad_area_base, mitad_area_lateral, mitad_area_total, mitad_volumen;

    // CONSTANTE
    const float PI_CONST = 3.14159;

    // LEER POR TECLADO
    printf("Ingrese el radio del cilindro: ");
    scanf("%f", &radio);

    printf("Ingrese la altura del cilindro: ");
    scanf("%f", &altura);

    // EVALUAR EXPRESION LOGICA
    if (radio > 0 && altura > 0) {

        // CALCULOS
        area_base = PI_CONST * radio * radio;
        area_lateral = 2 * PI_CONST * radio * altura;
        area_total = 2 * area_base + area_lateral;
        volumen = area_base * altura;

        // CALCULAR LAS MITADES DE LAS RESPUESTAS
        mitad_area_base = area_base / 2.0;
        mitad_area_lateral = area_lateral / 2.0;
        mitad_area_total = area_total / 2.0;
        mitad_volumen = volumen / 2.0;

        // PRESENTAR AL USUARIO
        printf("\n--- RESULTADOS DEL CILINDRO ---\n");
        printf("El area base es: %.4f\n", area_base);
        printf("La mitad area base es: %.4f\n", mitad_area_base);

        printf("El area lateral es: %.4f\n", area_lateral);
        printf("La mitad area lateral es: %.4f\n", mitad_area_lateral);

        printf("El area total es: %.4f\n", area_total);
        printf("La mitad area total es: %.4f\n", mitad_area_total);

        printf("El volumen es: %.4f\n", volumen);
        printf("La mitad_volumen es: %.4f\n", mitad_volumen);

    } else {
        printf("\nError: El radio y la altura deben ser mayores que 0.\n");
    }

    return 0;
}
```

Fig. 77. Programación en C del décimo noveno ejercicio.

Salida del programa

```
Problemas  Salida  Consola de depuración  Terminal

PS C:\Users\Usuario\Desktop\proyecto.c> cd "c:
Ingrese el radio del cilindro: 3
Ingrese la altura del cilindro: 5

--- RESULTADOS DEL CILINDRO ---
El area base es: 28.2743
La mitad area base es: 14.1372
El area lateral es: 94.2477
La mitad area lateral es: 47.1239
El area total es: 150.7963
La mitad area total es: 75.3982
El volumen es: 141.3716
La mitad_volumen es: 70.6858
PS C:\Users\Usuario\Desktop\proyecto.c>
```

Fig. 78. Salida del programa del décimo noveno ejercicio.

Conclusiones del algoritmo

Se utilizó la condición if (radio > 0 && altura > 0) como filtro para que no me de errores el programa y se pueda copilar de una buena manera el algoritmo.

Mediante la constante PI_CONST, el programa calcula con precisión las áreas y el volumen del cilindro, dividiendo luego cada resultado entre 2.0 para obtener sus mitades exactas.

EJ- 380

Enunciado del algoritmo

En una gasolinera, cree un algoritmo que simule un sistema de votación. Lea tres votos (enteros para candidatos A, B, C). Use constantes simbólicas VOTOS_MIN = 0, MAYORIA_ABSOLUTA = 0.5. Calcule el total de votos con (votos_a + votos_b + votos_c). Use expresiones relacionales y logicas para determinar el ganador. Evalúe las expresiones logicas: (votos_a > votos_b AND votos_a > votos_c), (votos_b > votos_a AND votos_b > votos_c), etc. Asigne el ganador a una variable cadena. Muestre por pantalla los resultados. Adicionalmente, calcule la mitad del resultado obtenido. Agregue un mensaje de bienvenida personalizado al iniciar el algoritmo.

Análisis del problema

Datos de entrada

Leer por teclado votos_a de tipo entero.

Leer por teclado votos_b de tipo entero

Leer por teclado votos_c de tipo entero.

Proceso

Constante VOTOS_MIN = 0, MAYORIA_ABSOLUTA = 0.5

Calcular el total de votos con (votos_a + votos_b + votos_c) asignar, en suma, como entero

Usar una serie de condicionales para evaluar quien tiene la mayoría de votos como (votos_a > votos_b AND votos_a > votos_c), (votos_b > votos_a AND votos_b > votos_c) asignar en cadena

Calcular la mitad de la suma de votos obtenidos asignar en mitad como real

Datos de salida

suma: la suma de los tres votos como entero

cadena: mostrar quien es el ganador en los votos obtenidos

mitad: la división para dos con la suma como real

Diagrama de flujo

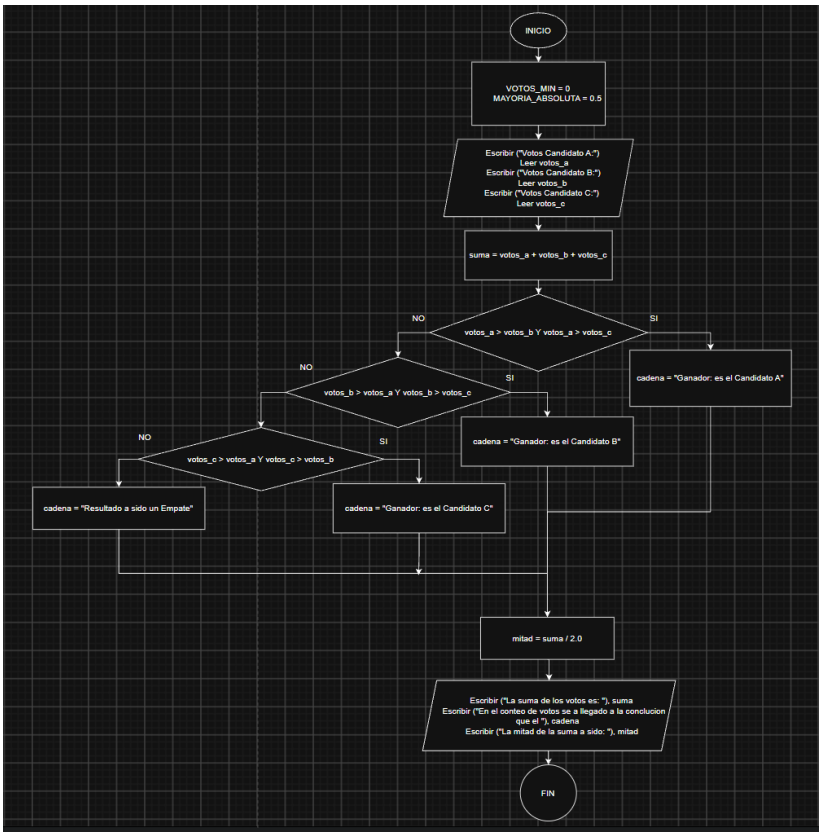


Fig. 79. Diagrama de flujo del vigésimo ejercicio.

Pseudocódigo

```
Algoritmo ultimo
    //DEFINICION DE VARIABLES
    Definir votos_a, votos_b, votos_c, suma Como Entero
    Definir mitad, MAYORIA_ABSOLUTA Como Real
    Definir VOTOS_MIN Como Entero
    Definir cadena Como Cadena

    // CONSTANTES
    VOTOS_MIN = 0
    MAYORIA_ABSOLUTA = 0.5

    // LECTUR POR TECLADO
    Escribir ("Votos Candidato A:")
    Leer votos_a
    Escribir ("Votos Candidato B:")
    Leer votos_b
    Escribir ("Votos Candidato C:")
    Leer votos_c

    // PROCESO
    suma = votos_a + votos_b + votos_c

    Si (votos_a > votos_b Y votos_a > votos_c) Entonces
        cadena = "Ganador: es el Candidato A"
    Sino
        Si (votos_b > votos_a Y votos_b > votos_c) Entonces
            cadena = "Ganador: es el Candidato B"
        Sino
            Si (votos_c > votos_a Y votos_c > votos_b) Entonces
                cadena = "Ganador: es el Candidato C"
            Sino
                cadena = "Resultaso a sido un Empate"
            FinSi
        FinSi
    FinSi

    mitad = suma / 2.0

    // PRESENTACION AL USUARIO
    Escribir ("La suma de los votos es: "), suma
    Escribir ("En el conteo de votos se a llegado a la conclucion que el "), cadena
    Escribir ("La mitad de la suma a sido: "), mitad
FinAlgoritmo
```

Fig. 80. Pseudocódigo del vigésimo ejercicio.

Prueba de escritorio

Paso	VOTOS_MIN	MAYORIA_ABSOLUTA	votos_a	votos_b	votos_c	suma	cadena	mitad	Proceso	Pantalla
1	0	0.5								
2			150						Lectura de variable votos_a	Votos Candidato A:
3				80					Lectura de variable votos_b	Votos Candidato B:
4					70				Lectura de variable votos_c	Votos Candidato C:
5						300			suma = 150 + 80 + 70	
6							Ganador: es el Candidato A		(150 > 80 Y 150 > 70) -> VERDADERO	
7								150	mitad = 300 / 2.0	
8									Impresión de resultados finales	La suma de los votos es: 300 En el conteo de votos se a llegado a la conclucion que el Ganador: es el Candidato A La mitad de la suma a sido: 150

Tabla 39. Primera prueba de escritorio del vigésimo ejercicio.

Paso	VOTOS_MIN	MAYORIA_ABSOLUTA	votos_a	votos_b	votos_c	suma	cadena	mitad	Proceso	Pantalla
1	0	0.5								
2			100						Lectura de variable votos_a	Votos Candidato A:
3				100					Lectura de variable votos_b	Votos Candidato B:
4					50				Lectura de variable votos_c	Votos Candidato C:
5						250			suma = 100 + 100 + 50	
6							Resultado a sido un Empate		Sin mayoría estricta entre A, B o C -> Empate	
7								125	mitad = 250 / 2.0	
8									Impresión de resultados finales	La suma de los votos es: 250 En el conteo de votos se a llegado a la conclucion que el Resultado a sido un Empate La mitad de la suma a sido: 125

Tabla 40. Segunda prueba de escritorio del vigésimo ejercicio.

Codificación en lenguaje C

```c
#include <stdio.h>
// DEFINICION DE VARIABLES
int main() {
int votos_a, votos_b, votos_c, suma , VOTOS_MIN;
float  mitad, MAYORIA_ABSOLUTA;
const char* cadena ;
// CONSTANTE
VOTOS_MIN = 0;
MAYORIA_ABSOLUTA = 0.5;
// LEER POR TECLADO
printf ("Votos del candidato A\n ");
scanf ("%d" , &votos_a);
printf ("Votos del candidato B\n ");
scanf ("%d" , &votos_b);
printf ("Votos del candidato C\n ");
scanf ("%d" , &votos_c);

//PROCESO
suma = votos_a + votos_b + votos_c;
mitad = suma * MAYORIA_ABSOLUTA;
// EVALUAR EXPRESION LOGICA
if (votos_a > votos_b && votos_a > votos_c){
cadena = ("Ganador: es el candidato A");
}else
    if (votos_b > votos_a && votos_b > votos_c){
    cadena = ("Ganador: es el candidato B");
    }else
        if (votos_c > votos_a && votos_c > votos_b){
        cadena = ("Ganador: es el candidato C");
        }
        else{
        cadena = ("Resultado a sido Empate");
        }

// PRESENTAR AL USUARIO
printf ("La suma de los votos es: %d \n" , suma);
printf ("En el conteo de votos se a llegado a la conclusion de que el %s \n" , cadena);
printf ("La mitad de la suma a sido: %.2f \n" , mitad);

return 0;
}
```

Fig. 81. Programación en C del vigésimo ejercicio.

Salida del programa

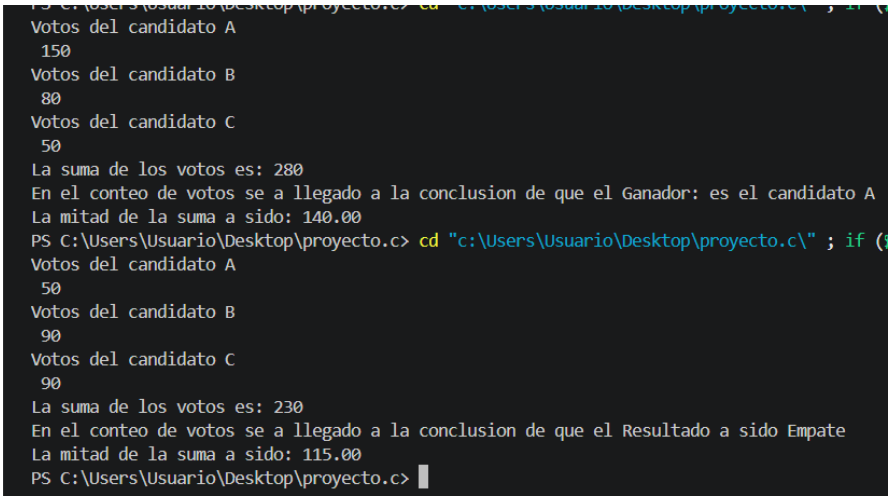


Fig. 82. Salida del programa del vigésimo ejercicio.

Conclusiones del algoritmo

Cuando se declaran variable tipo cadena se puede hacer utilizando un arreglo de caracteres (char cadena []), constantes de tipo cadena (const char*(vaiable)), o usando la librería <string.h> junto con la función strcpy para asignar el texto directamente dentro de los bloques condicionales de manera correcta para que no salga errores.

Gracias a las condiciones (Si-Sino), el programa pudo evaluar los votos de cada candidato para ver si existía un único ganador o si se presentaba un empate entre ellos.

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.

Firma del estudiante **Docente**